

UNIVERSITEIT•STELLENBOSCH•UNIVERSITY
jou kennisvennoot • your knowledge partner

Design of a universal cooperative collision avoidance system for autonomous vehicles using horizontal manoeuvres only.

by

Adrian Stark
25855670

Mechatronic Project 488

Project Report

Study leader: Prof JAA Engelbrecht

October 2026

Declaration

I, the undersigned, declare that the work contained in this report is my own original work.

Signature: .....
A. Stark

Date: .....

AI Use Declaration

1. I am convinced and can support my claim that my assessment product is an indication of my own learning, knowledge, skills, and understanding.
2. Where I have used AI tools for enhancing my own creation of ideas and words, I acknowledge that I have to declare it.
3. Where I have used AI tools for generating new ideas, words, code, image-prompts for other AI Image-generating tools, or structure and even presentations (or other AI tools that can be used as assistants to the knowledge building and representing process), I have declared and documented the use of such tools and I am prepared to talk about the process I used and what it contributed to my learning and insights.
4. I am aware that the lecturer can ask me to demonstrate my learning, for example through explaining the choices I made in terms of approach, content used, literature selected, conclusions drawn, etc. through an additional assessment like an oral (for example).
5. I understand that if I am not able to agree to the above points, there is a chance that my academic behaviour will be deemed unethical and might lead to a disciplinary case being brought against me on the grounds of cheating or plagiarism and that the standard procedures for such behaviour will be followed.
6. As per the Disciplinary Code of SU (par. 10.2.1 and 10.2.2), I understand that I take responsibility for the integrity of my work, which includes the obligation to ask for clarification from an academic member of staff if I am unsure of anything, and that I strictly adhered to all instructions received in the course of the academic assessment by relevant and authorized staff (whether the instruction is in oral or written format).
7. I understand that when I am not able to document and declare my use of AI tools, this behaviour will be deemed as cheating in examinations and assessments (Disciplinary Code 1.1 c.) as I referred to âunauthorized notes, books, electronic devices or other reference materialâ.

Signature:
Name: Student no:
Date:

EXECUTIVE SUMMARY

Student: A. Stark
Supervisor: Prof JAA Engelbrecht

Title of Project
Design of a universal cooperative collision avoidance system for autonomous vehicles using horizontal manoeuvres only.
Objectives
Model a universal cooperative collision avoidance system and demonstrate it with wheel-based robotic vehicles at scale to show its feasibility.
What is current practice and what are its limitations?
Currently, there are proprietary collision avoidance systems used by a variety of industries and companies, that means that small autonomous applications like consumer drones do not contain this technology. Also, due to their proprietary nature, most collision avoidance systems do not account for other vehicles routes, only addressing the issue as it becomes imminent.
What is new in this project?
This project will aim to create a universal system that can be used across a variety of autonomous vehicles, not designed for one specific case. It will also be cooperative, allowing the vehicles to work together to decide on the most efficient path for all.
If the project is successful, how will it make a difference?
If successful, this project will allow cooperative collision avoidance to be implemented in many more applications, and at any scale, lowering the barrier to entry for the use of this technology.
What are the risks to the project being a success? Why is it expected to be successful?
A model that meets the performance criteria cannot be created. If found, the model may not be demonstrable at small-scale. Time and budgetary risks are also present. All risks are mitigated by planning and making use of all available resources.
What contributions have/will other students made/make?
Previous students have considered cooperative collision avoidance in single applications. Their methods will be considered in the process of this project's completion.
Which aspects of the project will carry on after completion and why?
This project will prove that it is technically feasible, however it must be decided if it is economically feasible to continue and develop the product to deployable scale.
What arrangements have been/will be made to expedite continuation?
The chosen model/method and performance data will be provided. The project will be documented in sufficient detail that someone can scale it up to a deployable state.

Student	Date	Lecturer

Contents

Declaration i

Executive Summary iii

Contents iv

List of Figures vii

List of Tables ix

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Objectives . . . 1

2 Literature Review 3
- 2.1 Background . . . 3
- 2.2 Advantages to Cooperative Collision Avoidance . . . 3
- 2.3 Route Planning . . . 4
  - 2.3.1 A* Search . . . 4
  - 2.3.2 Dijkstra's Algorithm . . . 5
  - 2.3.3 Rapidly-Exploring Random Trees (RRT and RRT*) . . . 6
  - 2.3.4 Artificial Potential Fields . . . 7
  - 2.3.5 D* Lite . . . 8
- 2.4 Evaluation . . . 8
  - 2.4.1 Minimum Safe Distance . . . 9
  - 2.4.2 Path Length Comparison . . . 9
- 2.5 Software Dynamic Modelling . . . 10
  - 2.5.1 DC Motor Model . . . 10
  - 2.5.2 PID Discretisation . . . 11
- 2.6 Hardware for Demonstration . . . 12
  - 2.6.1 Tracking Options . . . 12
  - 2.6.2 Correcting Wide-Angle Lens Distortion . . . 13

3 System definition **14**
3.1 System architecture . . . 14
3.2 Operating environment . . . 14
3.3 Vehicle constraints . . . 14
3.4 Route planner and path following . . . 15
3.5 Assumptions . . . 15
3.6 Testing and evaluation . . . 16

4 Demonstration Design **17**
4.1 Demonstration Architecture . . . 17
4.2 Hardware Design . . . 17
4.2.1 Tracking system . . . 17
4.2.2 Vehicles . . . 18
4.2.3 Budget . . . 18
4.3 Interfacing . . . 20

5 Software Design **21**
5.1 Vehicle Software . . . 21
5.1.1 Communication . . . 21
5.1.2 Wheel speed estimation . . . 22
5.1.3 Wheel speed control . . . 22
5.1.4 Motor drive . . . 23
5.2 Path Follower . . . 23
5.2.1 Calibration and pose estimation . . . 23
5.2.2 Pure Pursuit . . . 24

6 Route planner selection **25**
6.1 Considerations and limitations . . . 25
6.2 Prioritized A* Search . . . 25
6.3 A* Search with no priority . . . 26
6.4 Collaborative A* Search . . . 26
6.4.1 Heuristic adjustments . . . 28
6.4.2 Cost function . . . 28
6.4.3 Well-clear distance . . . 29
6.4.4 Time and computation optimisation . . . 30

7 Vehicle modelling **32**
7.1 Motor Model . . . 32
7.1.1 Armature resistance . . . 32
7.1.2 Motor constant . . . 33
7.2 Vehicle Model . . . 33

7.2.1 Parameter fitting 34
7.2.2 Turning 34
7.3 Fleet Variation 34
7.4 Speed Controller Tuning 35

8 Simulation **39**
8.1 Wheel-speed loop 39
8.2 Vehicle 39
8.3 Path-following loop 40

9 Parameter setting **42**
9.1 Grid size 42
9.2 Look-ahead distance 42
9.3 Well-clear distance 42

10 Monte-Carlo Testing **45**
10.1 Setup 45
10.2 Success rate 45
10.3 Minimum separation 47
10.4 Cross-track error 47
10.5 Convergence 48
10.6 Worst-case scenario 48

A A* Search algorithm **51**
A.1 Well-clear distance formula derivation 51

B Dynamic straight line model derivation **53**

List of References **55**

List of Figures

1.2.1 Hardware setup 2

4.2.1 Undistorted overhead camera image of movement area 18
4.2.2 Robotic vehicle 19
4.3.1 Data flow between the laptop, camera and vehicles 20

5.1.1 Program flow of the vehicle firmware 21
5.1.2 Schematic of vehicle hardware connections 22

6.2.1 A* Search with 3 directions 26
6.3.1 A* Search with 5 directions and no priority 27
6.4.1 Collaborative (centralised joint) A* Search, planning all three vehicles simultaneously on a 4-connected grid 31

7.1.1 Locked-rotor test, vehicle 1 left motor 32
7.2.1 Dynamic and kinematic free-body diagrams of the vehicle 33
7.2.2 Vehicle 1, equal voltages on both wheels, with model 34
7.2.3 Vehicle 1, 5 V left and 4 V right, with model 35
7.3.1 Vehicle 2, equal voltages on both wheels, with vehicle 1 model 35
7.3.2 Vehicle 3, equal voltages on both wheels, with vehicle 1 model 36
7.4.1 Vehicle 1, straight-line closed-loop step response, with model 36
7.4.2 Vehicle 1, turning closed-loop step response, with model 37
7.4.3 Vehicle 2, straight-line closed-loop step response 37
7.4.4 Vehicle 3, straight-line closed-loop step response 38

8.1.1 Wheel-speed loop for one side of a vehicle (left side shown) 39
8.2.1 Vehicle model, including the ArUco tracking error and delays 40
8.3.1 Path-following loop for vehicle 1 41

9.3.1 Collision boundary and planning well-clear distance around a vehicle 43
9.3.2 Square test on the 400 mm grid with a 400 mm look-ahead 44

10.2.1 Outcomes of the 500 Monte-Carlo runs 46

10.2.2 Minimum separation achieved in each run 46
10.3.1 Delivered minimum separation for plans at the requirement and plans with margin above it 47
10.4.1 Peak cross-track error of each vehicle-run 48
10.5.1 Convergence of the failure rates and of the 95th-percentile cross-track error 49
10.6.1 Worst-case run: driven and commanded paths (left) and inter-vehicle separation (right) 50

List of Tables

3.3.1 System parameters for the demonstration 15

4.2.1 Project budget 19

Chapter 1
Introduction

1.1 Background

Most vehicles operate in a world in which autonomous and non-autonomously controlled vehicles must inhabit the same space. For example, both traditional cars and newer cars with partial-self driving must use the same roads. This has led to complex collision avoidance systems that react to the scenario directly in front of them. This means that vehicles can only respond reactively to their surroundings and not proactively. Reacting proactively will make these systems more effective. If the vehicles involved in each environment can work together to determine the least intrusive paths to their destinations, the need for reactive collision avoidance can be reduced or even negated. This does require all vehicles involved to be able to communicate with each other but there are scenarios where that is increasingly common, consider using fixed-wing mapping drones all flying over the same area, delivery drones servicing the same area from a distribution hub or carrier robots moving around on a factory floor. Since there are a variety of these scenarios, a universal cooperative collision avoidance system would make it easier for this technology to be effectively implemented in a system.

This project covers the creation and demonstration of a universal cooperative collision avoidance system for autonomous vehicles using horizontal manoeuvres only.

This project, which Mr Adrian Stark is doing as part of the Mechatronic Project 488 module, stems from a proposal by Prof JAA Engelbrecht. It plans to build on work done by previous master's students (Thando Mtimkulu and Merrick Hughes) who each investigated this technology for a specific use case.

This document is a detailed report that covers the project objectives, literature review, system definition, system design and modelling as well as testing and verification.

1.2 Objectives

This project's aim is to create a cooperative collision avoidance system for autonomous vehicles. The system will make use of the vehicles location, heading and desired destination to determine a new path for each vehicle to its destination with the smallest impact to all vehicles path lengths. The system will consider the

vehicles minimum velocities and maximum turn rates. This project will consider vehicles that cannot slow rapidly and need to maintain a minimum speed to be in operation. This is to ensure that the system can be used in applications like fixed wing aircraft or marine vessels. The project is limited to horizontal manoeuvres so the traditional z-dimension cannot be used as a collision avoidance tactic.

Figure 1.2.1 illustrates this project's intended outcome. It will create paths through a tracked movement area for each each vehicle to follow that ensure that the vehicles do not collide with each other. To achieve this the project has 3 main objectives:

1. Create a route planner to determine the paths for the vehicles.
2. Create a lumped-mass model of the system to evaluate system success.
3. Demonstrate feasibility on scaled vehicles.

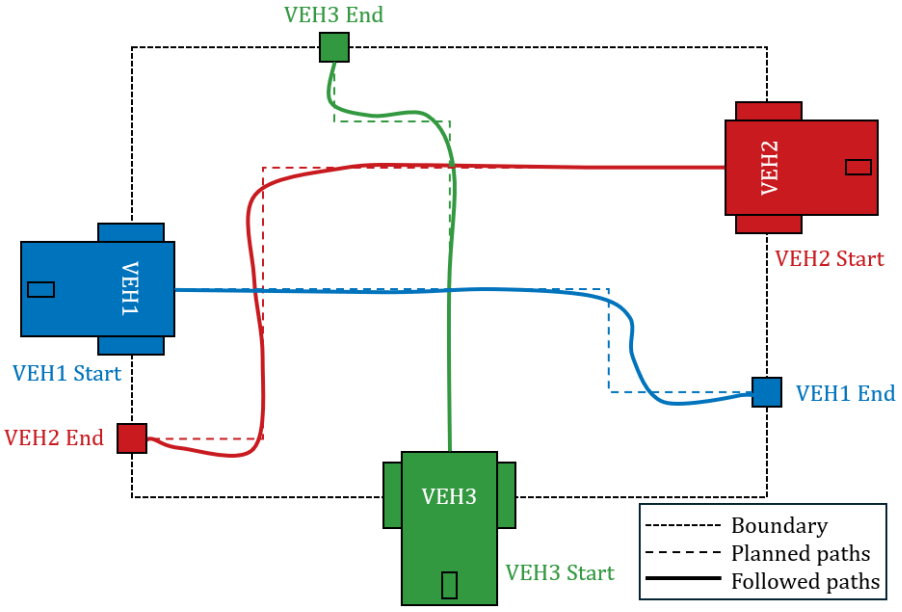


Figure 1.2.1: Hardware setup

Chapter 2
Literature Review

2.1 Background

Traditional collision avoidance is built on onboard sensing and reactive control. A vehicle senses its surroundings with radar, lidar, ultrasonic, or cameras and converts what it sees directly into a braking or steering command. On the road, this appears as adaptive cruise control and forward-collision warning with automatic emergency braking (Vahidi and Eskandarian, 2003). Most of the difficulty sits in detection, not control: recognising another vehicle has to hold up across changing light, weather, occlusion, and clutter, and every sensor trades range and resolution against cost (Mukhtar *et al.*, 2015).

Planning is usually reactive as well. The Dynamic Window Approach searches a vehicle's feasible velocities over a short horizon and picks whichever keeps it clear of what is nearby while making progress toward the goal (Fox *et al.*, 1997). This runs comfortably in real time, but it optimises over the current sensor snapshot only, with no model of where anything else will be a few seconds later.

Four problems follow from this arrangement. Each vehicle plans from its own point of view with no knowledge of what the others intend, so it has to assume the worst and drive conservatively, sometimes to the point of deadlock. A purely reactive planner can settle into a local minimum or oscillate near another vehicle instead of committing to a manoeuvre. Sensing range and line of sight mean a fast-approaching vehicle outside the field of view stays invisible until it is already close. And when two vehicles each react only to the other, with neither accounting for the fact that the other is also reacting, the result is a back-and-forth that neither one resolves. These four problems are the case for letting vehicles cooperate rather than plan alone.

2.2 Advantages to Cooperative Collision Avoidance

Cooperative collision avoidance (CCA) shares vehicle state and intent so that the work of avoiding a collision is split between vehicles instead of each one guessing at what the others will do.

Velocity-obstacle methods make this concrete. The velocity obstacle of another moving vehicle is the set of a vehicle's own velocities that would eventually put

it on a collision course. van den Berg *et al.* (2008) split this evenly between the two vehicles in Reciprocal Velocity Obstacles (RVO), which removes the oscillation that appears when two purely reactive agents both try to dodge in full. van den Berg *et al.* (2011) extend this to Optimal Reciprocal Collision Avoidance (ORCA), writing each pairwise constraint as a half-plane of permitted velocities and intersecting them across all neighbours at once; the result scales to large groups without pairwise oscillation. This is where the case for cooperation is clearest: with many vehicles close together, splitting the avoidance effort produces smoother, collision-free motion that an isolated reactive planner cannot match.

At an intersection, cooperative control uses vehicle-to-vehicle and vehicle-to-infrastructure communication to schedule vehicles through a shared conflict zone rather than leaving each one to brake reactively. Hafner *et al.* (2013) demonstrate this on real test vehicles, coordinating their passage to stay collision-free.

Cooperation is not always the better choice. It helps most in dense traffic, at structured conflicts such as intersections, in a fleet running one shared protocol, and wherever onboard sensing alone falls short. It performs worse when the communication link is the weak point: latency, dropped messages, and limited range all degrade coordination. It also assumes every relevant vehicle is cooperating and behaving predictably, so a single non-cooperating vehicle can void the guarantees, and the extra communication and coordination logic is itself a new source of complexity and failure.

2.3 Route Planning

CCA requires each vehicle to compute a route to its goal that stays clear of the other vehicles' current and predicted positions. Because the only hazards this project plans around are other moving vehicles, no method here needs to encode a fixed map of terrain; instead, every planner has to account for time as well as space, since a given position is only in the way at a particular instant, not permanently. A*, Dijkstra, RRT and RRT*, artificial potential fields, and D* Lite are the methods most often used for this, and each handles the time dimension differently: A* and Dijkstra search a graph or grid and need it extended with a time axis to represent motion; RRT samples a continuous space and can resample around a moving constraint as it shifts; potential fields react continuously to the other vehicles' current positions and update automatically as they move; and D* Lite repairs an existing plan incrementally as the other vehicles' predicted paths change the cost of a given cell at a given time, rather than solving again from scratch.

2.3.1 A* Search

A* keeps two sets of nodes: an open set of candidates still to be explored, ordered by estimated total cost, and a closed set of nodes already finalised. At each step it

removes the open-set node with the lowest

$$f(n) = g(n) + h(n), \tag{2.3.1}$$

where $g(n)$ is the cost of the best path found so far from the start to n and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal, expands its neighbours, and updates g for any neighbour reached more cheaply through n. Provided $h(n)$ never overestimates the true remaining cost, the path found the first time the goal is removed from the open set is optimal (Hart *et al.*, 1968). On a grid the heuristic is usually the straight-line distance, $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$, or, where only axis-aligned moves are allowed, the Manhattan distance, $h(n) = |x_n - x_g| + |y_n - y_g|$, with (x_g, y_g) the goal cell.

Plain A* searches a fixed map and says nothing about when a cell is occupied. Because the only hazards considered here are other vehicles, each following a predicted trajectory, the search has to run over states (x, y, t) rather than (x, y): a reservation table records which cells are claimed by another vehicle at each future time step, and a neighbour is only expanded if its cell is still free at the time the planning vehicle would reach it (Silver, 2005). This space-time form of A*, often called cooperative A*, is the version that actually applies to a system built to avoid other vehicles rather than fixed terrain.

Advantages: optimal and complete with an admissible heuristic; the heuristic biases the search toward the goal, so far fewer nodes are expanded than an uninformed search would need; straightforward to implement; and weighting the heuristic trades optimality for speed when that is worth doing (Duchoň *et al.*, 2014).

Drawbacks: memory and run time both grow with grid resolution and, once time is added as a third dimension, with the length of the planning horizon; paths follow grid edges and turn at fixed angles, so they usually need smoothing before a wheeled vehicle can track them; and a change to another vehicle's predicted trajectory invalidates the reservation table and forces a fresh search (Duchoň *et al.*, 2014).

2.3.2 Dijkstra's Algorithm

Dijkstra's algorithm is the special case of A* with $h(n) = 0$: every node is expanded purely by its known cost from the start (Dijkstra, 1959). It keeps a priority queue of nodes ordered by $g(n)$ and repeatedly relaxes every edge (u, v) with cost $w(u, v)$ out of the node it removes:

$$\text{if } g(u) + w(u, v) < g(v) \quad \text{then} \quad g(v) \leftarrow g(u) + w(u, v), \tag{2.3.2}$$

re-inserting v with its updated cost. With a binary heap this runs in $O((|V| + |E|) \log |V|)$ for a graph with $|V|$ nodes and $|E|$ edges, and because it carries no heuristic, it explores outward from the start roughly evenly in every direction rather than toward any one goal.

Advantages: it is guaranteed to return the shortest path; it needs no heuristic to be designed or tuned; and because it expands uniformly it produces shortest paths to every node reached, not only one goal, which is useful when several destinations are queried against the same map (Patle *et al.*, 2019).

Drawbacks: with no heuristic to bias it toward a single goal, it typically expands many more nodes than A* for a single start-goal query, which costs time on a large map; and, like A*, a static solve says nothing about when a cell will be occupied, so it inherits the same need to be extended with a time dimension, or re-run whenever the other vehicles' predicted paths change (Patle *et al.*, 2019).

2.3.3 Rapidly-Exploring Random Trees (RRT and RRT*)

RRT builds a tree outward from the start by sampling the configuration space rather than discretising it (LaValle, 1998). Each iteration draws a random sample x_{rand}, finds the tree's nearest existing node x_{near} under a distance metric, and steers from x_{near} toward x_{rand} by at most a fixed step η to produce a candidate node x_{new}; the candidate is added to the tree only if the segment connecting the two is collision-free. Repeating this biases the tree toward unexplored regions of the space, and it terminates once a sample falls within reach of the goal.

RRT* adds a rewiring step so the plan converges toward the optimal path as sampling continues (Karaman and Frazzoli, 2011). Instead of attaching x_{new} to its single nearest neighbour, it considers every existing node within a shrinking radius

$$r = \min\left\{\gamma\left(\frac{\ln n}{n}\right)^{1/d},\ \eta\right\}, \tag{2.3.3}$$

where n is the current number of nodes in the tree, d is the dimension of the configuration space, γ is a constant set by the free-space volume, and η is the maximum step length as before. Among the nodes in this radius, x_{new} is connected to whichever gives it the lowest path cost

$$c(x_{\text{new}}) = c(x_{\text{parent}}) + \|x_{\text{new}} - x_{\text{parent}}\|, \tag{2.3.4}$$

and any neighbour that would itself become cheaper by routing through x_{new} is rewired to do so. Because the only hazards here are other vehicles, the collision check applied to each candidate segment tests it against their predicted positions over the time the vehicle would take to traverse it, rather than against any fixed geometry.

Advantages: both handle high-dimensional and continuous spaces without building a grid; both extend naturally to a vehicle's real turning and acceleration limits by using a steering function that respects those limits instead of straight-line segments; and RRT* converges toward the optimal path as the sample count grows (Karaman and Frazzoli, 2011).

Drawbacks: the raw tree is jagged and usually needs smoothing before it is drivable; plain RRT is feasible but not optimal; RRT*'s rewiring adds a nearest-neighbour and cost-comparison step at every iteration, so convergence to a good path can be slow; and because sampling is random, two runs over the same problem rarely produce the same path (LaValle, 1998).

2.3.4 Artificial Potential Fields

The artificial potential field treats the goal as an attractor and every other vehicle as a repeller, and moves the vehicle down the gradient of their combined potential (Khatib, 1986):

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q), \tag{2.3.5}$$

$$U_{\text{att}}(q) = \tfrac{1}{2}\,\zeta\,\|q - q_{\text{goal}}\|^2, \tag{2.3.6}$$

$$U_{\text{rep}}(q) = \begin{cases} \tfrac{1}{2}\,\eta\left(\dfrac{1}{\rho(q)} - \dfrac{1}{\rho_0}\right)^2, & \rho(q) \le \rho_0, \\ 0, & \rho(q) > \rho_0, \end{cases} \tag{2.3.7}$$

$$F(q) = -\nabla U(q), \tag{2.3.8}$$

where q is the vehicle's own position, q_{goal} its goal, ζ the attractive gain, η the repulsive gain, $\rho(q)$ the distance to the nearest other vehicle, and ρ_0 the range within which that vehicle has any influence at all. The resulting force is converted directly into a steering command at every control step,

$$q_{k+1} = q_k + \alpha\,F(q_k), \tag{2.3.9}$$

with α a step-size gain and k the control-loop index. Because $\rho(q)$ is measured to the other vehicles' current positions, the field updates on its own as they move, with no separate replanning step.

Advantages: cheap to compute and continuous, so it runs comfortably at the control-loop rate rather than needing a discrete search; it reacts to other vehicles as they move with no replanning logic; and the resulting motion is smooth (Khatib, 1986).

Drawbacks: the vehicle can stall in a local minimum where the attractive and repulsive terms cancel, and it can oscillate when squeezed between two vehicles on either side; a goal positioned close to another vehicle's repulsive field can become unreachable; and because the force depends only on the present configuration, it carries no notion of where a vehicle is heading, only where it currently is. Modified potential functions that add an extra term to remove this goal-near-vehicle failure, and rotational terms to help escape local minima, address some of these cases (Ge and Cui, 2000).

2.3.5 D* Lite

D* Lite is an incremental planner for costs that change while a vehicle is already moving toward its goal. Rather than solving the graph again from scratch every time another vehicle's predicted path changes the cost of using a particular cell at a particular time, it repairs only the part of the existing solution that the change actually affects (Stentz, 1994). It keeps two values for every state s: $g(s)$, the current estimate of the cost from s to the goal, and a one-step lookahead value (Koenig and Likhachev, 2002)

$$\mathrm{rhs}(s) = \begin{cases} 0, & s = s_{\mathrm{goal}}, \\ \min\limits_{s' \in \mathrm{Succ}(s)} \big(c(s, s') + g(s')\big), & \text{otherwise}, \end{cases} \tag{2.3.10}$$

where $c(s, s')$ is the cost of moving from s to a successor s', itself raised wherever another vehicle is predicted to occupy s' at the relevant time. A state is *consistent* once $g(s) = \mathrm{rhs}(s)$; any state where the two disagree is queued for re-expansion, ordered by the key

$$k(s) = \Big[\ \min\big(g(s), \mathrm{rhs}(s)\big) + h(s_{\mathrm{start}}, s) + k_m\ ;\ \min\big(g(s), \mathrm{rhs}(s)\big)\ \Big], \tag{2.3.11}$$

compared alphabetically, where $h(s_{\mathrm{start}}, s)$ is the heuristic distance from the vehicle's current state to s and k_m is an offset that accumulates as the vehicle moves so that keys computed earlier stay comparable to new ones. When another vehicle's predicted trajectory changes, only the costs $c(s, s')$ it touches are updated, the affected states are re-queued, and expansion resumes from the front of the queue rather than from the start node — which is what makes the repair cheap.

Advantages: it repairs a plan quickly as the other vehicles' predicted trajectories are updated, which is the recurring event in this project rather than the exception; and it reuses the previous solve instead of discarding it, which is the main saving over rerunning A* every time a prediction changes (Koenig and Likhachev, 2002).

Drawbacks: it is more involved to implement and keep correct than a plain A* search, since consistency has to be maintained across incremental updates rather than computed once; it is still a grid method and inherits the same path-smoothness problems; and it carries a memory overhead for the stored priority queue and state values between updates (Stentz, 1994).

2.4 Evaluation

Two questions decide whether a route-planning method is fit for CCA: does it keep every pair of vehicles at a safe separation, and how much extra distance does avoidance cost compared with driving straight to the goal. The methods in Section 3 are compared against both criteria over the same set of test trajectories.

2.4.1 Minimum Safe Distance

For vehicles i and j with positions $p_i(t), p_j(t) \in \mathbb{R}^2$, the separation at time t is

$$d_{ij}(t) = \|p_i(t) - p_j(t)\|. \tag{2.4.1}$$

A trial is collision-free only if $d_{ij}(t) \geq d_{\text{safe}}$ for every pair and every t during the run. The safe distance is not a single fixed number; it is built from the vehicles' physical size and their ability to stop or turn away:

$$d_{\text{safe}} = r_i + r_j + v_{\text{rel}}\, t_r + \frac{v_{\text{rel}}^2}{2\, a_{\max}}, \tag{2.4.2}$$

where r_i and r_j are the bounding radii of the two vehicles, $v_{\text{rel}} = \|v_i(t) - v_j(t)\|$ is their relative speed, t_r is the combined sensing-and-actuation reaction time, and $a_{\max}$ is the maximum deceleration either vehicle can apply. The last two terms are the same reaction-distance-plus-braking-distance construction used in adaptive cruise control headway design (Vahidi and Eskandarian, 2003), applied here between two moving vehicles rather than between a vehicle and the one ahead of it. The minimum separation actually achieved during a run,

$$d_{\min} = \min_{t} \min_{i \neq j} d_{ij}(t), \tag{2.4.3}$$

is recorded for every trial and every planner; $d_{\min} \geq d_{\text{safe}}$ is a pass, and anything below it is a near-miss or collision. Because both the RVO and ORCA formulations already define a collision in terms of the combined radii of the two vehicles (van den Berg *et al.*, 2008, 2011), this criterion is directly compatible with cooperative planners evaluated the same way.

2.4.2 Path Length Comparison

Safety alone does not capture how much a planner disturbs a vehicle's route. Because the only hazards in this project are other vehicles, the shortest possible path for a vehicle acting alone is simply the straight line to its goal,

$$L_0 = \|p_{\text{goal}} - p_{\text{start}}\|. \tag{2.4.4}$$

The length of the path a vehicle actually drives once it has to avoid the other vehicles is the arc length of its logged trajectory,

$$L = \int_{t_0}^{t_f} \|\dot{p}(t)\|\, dt = \int_{t_0}^{t_f} \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\, dt, \tag{2.4.5}$$

where $p(t) = (x(t), y(t))$ is the vehicle's position and t_0, t_f are the start and end times of the run. Simulink and the tracking hardware in Section 6 both produce

sampled data rather than a continuous function, so L is evaluated as a discrete sum over the N logged samples,

$$L \approx \sum_{k=0}^{N-1} \|p_{k+1} - p_k\|. \tag{2.4.6}$$

The difference between the two,

$$\Delta L = L - L_0, \tag{2.4.7}$$

is the extra distance the avoidance manoeuvre cost, and the ratio $\eta = L_0/L$ gives a planner-independent efficiency score between 0 and 1, with $\eta = 1$ meaning the vehicle reached its goal without deviating at all. Running the same start/goal pairs and the same set of other-vehicle trajectories through each planner in Section 3 and comparing their $d_{\min}$ and η values side by side — together with the planning and replanning times already discussed for each method — is what decides which one is actually used.

2.5 Software Dynamic Modelling

The wheeled vehicles are driven by DC motors, with each wheel's speed regulated by a digital PID controller. Both are modelled in Simulink, working up from the motor to the discrete controller that drives it.

2.5.1 DC Motor Model

A brushed DC motor is described by one electrical and one mechanical equation (Kluever, 2019). The armature circuit gives

$$V_a(t) = R_a\, i_a(t) + L_a \frac{di_a(t)}{dt} + K_e\, \omega(t), \tag{2.5.1}$$

where V_a is the applied armature voltage, R_a and L_a are the armature resistance and inductance, i_a is the armature current, ω is the rotor angular velocity, K_e is the back-EMF constant, and $K_e\,\omega$ is the back-EMF opposing the applied voltage. This current produces a torque

$$T_m(t) = K_t\, i_a(t), \tag{2.5.2}$$

with K_t the torque constant, and the mechanical side — rotor plus load — follows

$$J \frac{d\omega(t)}{dt} + b\,\omega(t) = T_m(t) - T_L(t), \tag{2.5.3}$$

where J is the combined inertia, b the viscous friction coefficient, and T_L the load torque. In consistent SI units the torque and back-EMF constants are equal, $K_t =$

$K_e = K$. Taking Laplace transforms with $T_L = 0$ and eliminating i_a gives the voltage-to-speed transfer function used directly in Simulink,

$$\frac{\omega(s)}{V_a(s)} = \frac{K}{(L_a s + R_a)(Js + b) + K^2}, \tag{2.5.4}$$

(Kluever, 2019; Franklin *et al.*, 2019). Where L_a is small relative to the mechanical time constant it is often dropped, reducing the model to first order.

2.5.2 PID Discretisation

The continuous PID controller is

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad C(s) = K_p + \frac{K_i}{s} + K_d s, \tag{2.5.5}$$

where $e(t) = r(t) - y(t)$ is the error between the reference r and the measured output y, and K_p, K_i, K_d are the proportional, integral, and derivative gains (Franklin *et al.*, 2019). A digital implementation runs at a fixed sample period T, replacing the operator s with a discrete approximation. The common substitutions are

$$\text{Forward Euler:} \quad s \leftarrow \frac{z-1}{T}, \tag{2.5.6}$$

$$\text{Backward Euler:} \quad s \leftarrow \frac{z-1}{Tz}, \tag{2.5.7}$$

$$\text{Tustin (bilinear):} \quad s \leftarrow \frac{2}{T}\,\frac{z-1}{z+1}. \tag{2.5.8}$$

Applying backward differences to the integral and derivative terms gives the positional difference equation

$$u[k] = K_p\, e[k] + K_i T \sum_{j=0}^{k} e[j] + \frac{K_d}{T}\big(e[k] - e[k-1]\big), \tag{2.5.9}$$

and differencing this once more gives the incremental form actually run on the controller,

$$\Delta u[k] = K_p\big(e[k] - e[k-1]\big) + K_i T\, e[k] + \frac{K_d}{T}\big(e[k] - 2e[k-1] + e[k-2]\big), \tag{2.5.10}$$

where k is the sample index. The incremental form avoids storing the running error sum and handles integrator windup and controller switching more cleanly, which is why it is the common choice in embedded code (Franklin *et al.*, 2019). Tustin's substitution is usually preferred for the integral and derivative terms because it maps the entire left-half s-plane inside the unit circle, so a continuous design that is stable stays stable once discretised (Franklin *et al.*, 2019). Choosing the sample period T relative to the motor's own time constants is what keeps the discrete controller a faithful match to the continuous design it was tuned from (Kluever, 2019).

2.6 Hardware for Demonstration

Testing the collision avoidance system needs each vehicle's pose — position and heading — in a shared reference frame, in real time. The options span dedicated motion capture, camera-based fiducial markers, radio ranging, and onboard odometry.

2.6.1 Tracking Options

Motion capture (Vicon, OptiTrack). Several infrared cameras track reflective markers on each vehicle and reconstruct its pose to sub-millimetre accuracy at a high update rate (Merriaux *et al.*, 2017), which is why such systems are the usual choice for ground truth. The cost is in the cameras themselves, a fixed and carefully calibrated capture volume, and reflective markers fitted to every vehicle. This is expensive and not portable, and more infrastructure than a small demonstration generally needs.

ArUco fiducial markers. A single overhead camera detects printed square markers, and each marker's identifier and four corners together give location and orientation. ArUco's marker dictionaries are chosen so that valid patterns sit far apart in bit-pattern space, giving reliable identification with built-in error correction, and detection holds up under partial occlusion (Garrido-Jurado *et al.*, 2014); a later revision of the detector cuts detection time enough for real-time use (Romero-Ramirez *et al.*, 2018). The method is cheap, printed paper and one camera, and because every marker carries an identifier , several vehicles are told apart automatically. Its accuracy depends on camera resolution, marker size, and lighting; the pose is noisier than motion capture; corner detection degrades under motion blur and at shallow viewing angles; and it needs an unbroken line of sight. In practice, ArUco offers an open-source module inside OpenCV (`cv2.aruco`) with maintained ROS drivers, so marker generation, camera calibration, and pose estimation reduce to a handful of function calls, the main reason to choose it for a demonstration system.

AprilTag fiducial markers. AprilTag works on the same principle with a different tag family. Its detector has a very low false-positive rate (Olson, 2011), and a second version improves both detection speed and range (Wang and Olson, 2016). Detection is robust, accurate, and open source. Against that, the original version was heavier to compute than ArUco (the second version narrows that gap), a tag encodes fewer identifier bits for a given physical size, and it shares ArUco's line-of-sight and lighting constraints.

Ultra-wideband ranging. Fixed anchors and a tag on each vehicle measure signal time-of-flight and trilaterate position (Alarifi *et al.*, 2016). This needs no camera line of sight, is unaffected by lighting, and covers a larger area than a single overhead camera can. Its drawbacks: a single tag gives position but not heading, so a second tag or sensor fusion is needed; accuracy sits at the decimetre level; indoor multipath introduces error; and every anchor and tag adds hardware cost and its own calibration.

Wheel encoders. Integrating wheel-encoder counts gives a pose estimate with no external infrastructure at all, at a high rate and low cost. Because it integrates, wheel slip and small measurement errors accumulate into unbounded drift, and it carries no absolute reference. It is therefore used to smooth and fill in between updates from one of the methods above, not as the sole source of pose.

2.6.2 Correcting Wide-Angle Lens Distortion

An overhead camera wide enough to cover the whole test area typically uses a wide-angle, short-focal-length lens, which introduces strong radial (barrel) distortion: straight lines bow outward and marker corners shift, most severely near the edges of the frame, biasing any pose estimate taken there. The fix is camera calibration: estimating the lens parameters once and undistorting every frame before marker detection runs. The Brown–Conrady model relates an ideal, normalised image point (x, y) to the distorted point (x_d, y_d) the sensor actually records:

$$x_d = x\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \tag{2.6.1}$$

$$y_d = y\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \tag{2.6.2}$$

with $r^2 = x^2 + y^2$. The coefficients k_1, k_2, k_3 describe radial distortion and p_1, p_2 describe tangential distortion from lens misalignment. Zhang's method fits these coefficients, together with the camera's intrinsic matrix, from several views of a planar checkerboard (Zhang, 2000), and OpenCV implements both the calibration routine and the inverse mapping that undistorts each frame (Bradski, 2000). Calibrating once and undistorting every frame afterward removes most of the edge-of-frame error and keeps the ArUco pose estimate consistent across the whole field of view. For an extreme fisheye lens a dedicated fisheye model replaces the polynomial above.

Chapter 3

System definition

This chapter defines the system's components, operating environment, vehicle constraints and evaluation criteria. The constraints are expressed as vehicle parameters so that the planner can be tuned to other vehicle types; Table 3.3.1 gives the values used in the demonstration.

3.1 System architecture

The system has three components: a route planner that computes a collision-free path for every vehicle in the movement area; a Simulink model of the scaled vehicles that predicts how closely they track those paths; and a demonstration on scaled differential-drive robots, with a camera tracking an ArUco marker on each robot to measure its pose. The planned paths are the baseline for comparing the simulated and measured performance.

3.2 Operating environment

The vehicles operate in a rectangular movement area in the x-y plane. Each vehicle enters on one boundary and can exit on any boundary, so its path in the area is a segment of a longer route. The area represents the region where the vehicles routes converge and a collision is most likely. Practical and financial constraints limit the project to three vehicles. Changes in the z-direction are not available as an avoidance manoeuvre.

3.3 Vehicle constraints

Each vehicle is described by a minimum speed $v_{C,min}$ and a maximum turn rate $\omega_{C,max}$. Vehicles cannot stop or reverse ($v_{C,min} > 0$), as with fixed-wing aircraft and marine vessels that need forward speed to remain controllable. Each vehicle holds the same constant speed $v_C = v_{C,min}$ and avoids conflicts by changing its path rather than its speed. Its minimum turn radius is therefore fixed at:

$$r_{min} = \frac{v_C}{\omega_{C,max}} \tag{3.3.1}$$

and is set by of the each wheel's minimum angular velocity of $\omega_{wheel,min} = 5 rad/s$. Vehicles that can stop, hover or reverse can still use the planner, but it does not exploit those manoeuvres.

The differential-drive vehicles used can turn on the spot, so these limits are imposed by holding the linear velocity at v_C and saturating the turn rate at $\pm\omega_{C,max}$ both done by limiting the left and right wheel speeds.

Table 3.3.1: System parameters for the demonstration

Parameter	Symbol	Value
Movement area	$L \times W$	$2.4 \times 1.6\ m$
Number of vehicles	N	≤ 3
Minimum speed	$v_{C,min}$	0.25 m/s
Maximum turn rate	$\omega_{C,max}$	1.29 rad/s (or 74 $^\circ/s$)

3.4 Route planner and path following

For each vehicle, the planner takes the entry pose (x_0, y_0, ψ_0), the exit pose (x_f, y_f, ψ_f), the speed v_C and the turn-rate limit $\omega_{C,max}$, and outputs an ordered vector of x-y waypoints. It minimises the total path length of the fleet subject to a well-clear distance, d_{min}:

$$\min \sum_{i=1}^{N} L_i \quad \text{subject to} \quad \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \geq d_{min} \quad \forall\, t,\ i \neq j, \tag{3.4.1}$$

where L_i is the length of path i, which at constant speed is proportional to the vehicle's travel time, and $\mathbf{p}_i(t)$ is the position of vehicle i. The well-clear distance covers the vehicle footprint plus margins for path-tracking and other errors.

Each vehicle follows its waypoints with a Pure Pursuit controller, which converts a look-ahead point on the path into a turn-rate command.

3.5 Assumptions

- A single, centralised planner computes all paths before the vehicles enter the movement area. This allows route planner computation time to be neglected, but can only be done as it is performed centrally on a laptop, not in a distributed computing network.
- Collisions can occur only within the movement area.
- The vehicles have no proximity sensing. As the planner knows every vehicle's pose, destination and constraints, sensing is not needed to resolve conflicts, and omitting it reduces the cost of the demonstration. A real-world implementation would still need to identify which vehicles are converging on the area.

3.6 Testing and evaluation

The planner is tested on randomised entry and exit poses and on 3 edge cases: two vehicles on a head-on collision course, three vehicles merging to the same destination and vehicles crossing perpendicular to each other. A run is successful if every vehicle reaches its destination and no two vehicles come into contact with each other. The success rate over the randomised scenarios must exceed 99%. A run maintains well-clear if every vehicle remains at least d_{min} whilst in motion within the movement area. This well-clear requirement must be met by at least 75% of the randomised runs. Hardware runs are limited, so this rate is evaluated in simulation, and the demonstration verifies the models performance is indicative of the demonstrated system. The system will be demonstrated on the key scenarios of the edge-cases and scenarios in which collisions took place.

Chapter 4
Demonstration Design

This chapter describes how the planned paths are executed on scaled vehicles. The demonstration is scoped to be achievable within the time and budget of a final-year project, and this drives the centralised architecture.

4.1 Demonstration Architecture

Cooperation in the demonstration is centralised. The vehicles do not communicate with each other or broadcast their poses. Instead, a laptop plans the paths for all vehicles, tracks their poses with an overhead camera and sends each vehicle the commands needed to follow its path. Each vehicle's microcontroller only regulates its wheel speeds. Centralising the computation reduces the hardware on each vehicle and keeps the demonstration within budget, while still testing the planner itself. Applications without a central ground station would need to distribute this computation across the vehicles.

4.2 Hardware Design

4.2.1 Tracking system

The tracking system was chosen for the planar nature of the problem and the budget. Of the options reviewed in Section 2.6.1, a single overhead camera tracking ArUco markers gives the full pose of every vehicle at low cost, with no additional sensors on the vehicles.

The camera is a USB module with a Sony IMX179 sensor and a 145° diagonal field-of-view lens. It uses the USB Video Class (UVC) protocol, so it works with standard operating-system drivers. It is mounted on the ceiling $2.45m$ above the floor and streams 1920×1080 pixel frames at 30 frames per second. Each vehicle carries an ArUco marker with a side length of 100mm, from which its position and heading are estimated, and four further markers on the boundary of the movement area provide the reference frame for the vehicle poses. The wide-angle lens causes strong barrel distortion, so the camera's intrinsic parameters and distortion coefficients are calibrated before use by the Brown-Conrady method.

At this height, the camera covers a theoretical floor area of 14.15×7.96 m, found from $2h\tan(\theta/2)$ using the horizontal and vertical fields of view.

Ground resolution falls rapidly towards the edges of the image, so the movement

area is limited to 2.4×1.6 m, within which each marker spans at least TBD pixels. The floor space available for testing imposes a similar limit. Figure 4.2.1 shows the setup.

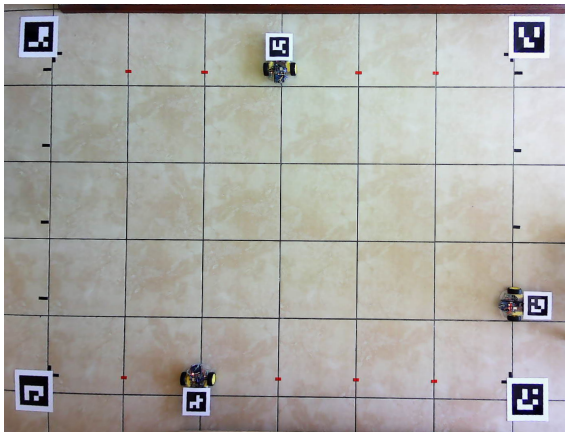


Figure 4.2.1: Undistorted overhead camera image of movement area

4.2.2 Vehicles

Each vehicle is built on a 2WD robot kit that provides an Arduino Uno R3 (ATmega328P), two DC gear motors, an L298N dual H-bridge driver, wheels and a chassis. Using a kit keeps the cost low and gives three vehicles of the same design. Each vehicle is powered by a 2S Li-Po battery (7.4 V nominal, 8.4 V fully charged).

The two driven wheels have separate motors, allowing each vehicle to be differentially-driven. An optical encoder on each wheel measures its speed, and a PI controller on the Arduino regulates each wheel's speed through the PWM duty cycle every 20 ms. The PWM frequency is raised to about 31 kHz so that the motor current conducts continuously, as the averaged-voltage motor model in Chapter 6 assumes. The laptop sends wheel-speed references to each vehicle through an HC-06 Bluetooth serial module. Figure 4.2.2 shows a vehicle.

4.2.3 Budget

The budget for final-year projects in the Department of Mechanical and Mechatronic Engineering is R5000. Components were chosen to stay within this limit while leaving a contingency for failures and unplanned purchases. Both occurred: extra motors were bought for testing, and the Li-Po balance charger failed and had to be replaced. The total component cost, listed in Table 4.2.1, is R5020.48, which exceeds the budget by R20.48. This extra cost was cleared with Prof JAA Engelbrecht.

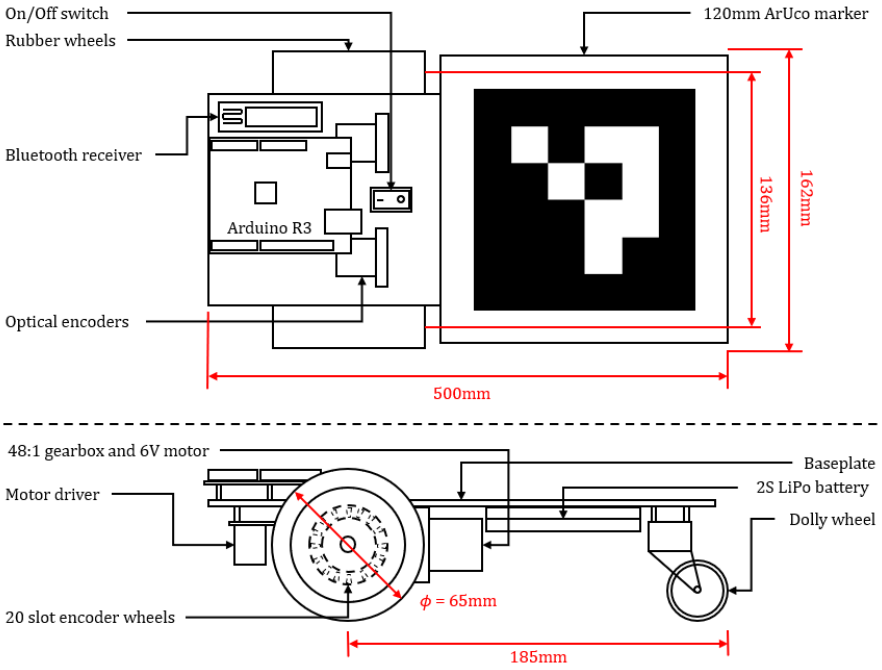


Figure 4.2.2: Robotic vehicle

Table 4.2.1: Project budget

Qty	Component	Unit price (R)	Subtotal (R)
3	2WD robot base	425.22	1275.65
3	Bluetooth receiver	99.13	297.39
6	Wheel encoder	21.74	130.44
1	Camera	798.00	798.00
9	Li-Po battery	128.00	1152.00
2	Li-Po charger	404.00	808.00
2	USB extension (1.5 m)	29.00	58.00
1	JST-XH connector kit	58.00	58.00
1	12 V 5 A AC adapter	146.00	146.00
1	22 AWG wire (5 m)	59.00	59.00
3	Extra motor and wheel kit	38.00	114.00
1	Printing and laminating	124.00	124.00
		Total	**5020.48**

4.3 Interfacing

Figure 4.3.1 shows the data flow between the laptop, camera and vehicles. A run proceeds as follows:

1. The vehicles are placed in the movement area. Their locations are measured off the set reference of the bottom left ArUco marker using a tape measure.

2. The planner computes the collision-free paths and writes each vehicle's waypoints to a `.csv` file.

3. During the run, the MATLAB ArUco detection library converts each camera frame into a pose (x, y, ψ) for every vehicle. The control script passes each pose to a Pure Pursuit controller, which computes the linear velocity and turn rate needed to follow the path; the script then converts these into left and right wheel-speed references.

4. The laptop sends the references to each vehicle over Bluetooth, and the Arduino's PI controllers track them using encoder feedback.

5. This process continues until each vehicle is at it's destination, where the vehicle is stopped.

The control is therefore split into two nested loops: an outer path-following loop on the laptop, closed through the camera at its frame rate of 30fps, and an inner wheel-speed loop on each Arduino, closed through the encoders at 16.67 Hz.

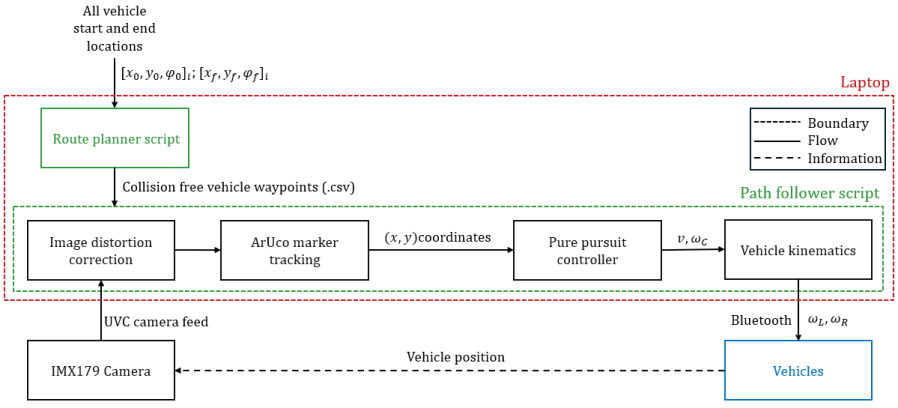


Figure 4.3.1: Data flow between the laptop, camera and vehicles

Chapter 5
Software Design

The software is split between the vehicles, which regulate their wheel speeds, and the laptop, which closes the path-following loop through the camera (Chapter 4).

5.1 Vehicle Software

Each vehicle drives its wheels at the speeds last commanded by the laptop. The vehicle firmware has a fixed tick of $T = 60$ ms: interrupt service routines timestamp the encoder edges, and each tick estimates both wheel speeds, updates the PI controllers and sets the motor duty cycles. Figure 5.1.1 shows the program flow and Figure 5.1.2 the wiring.

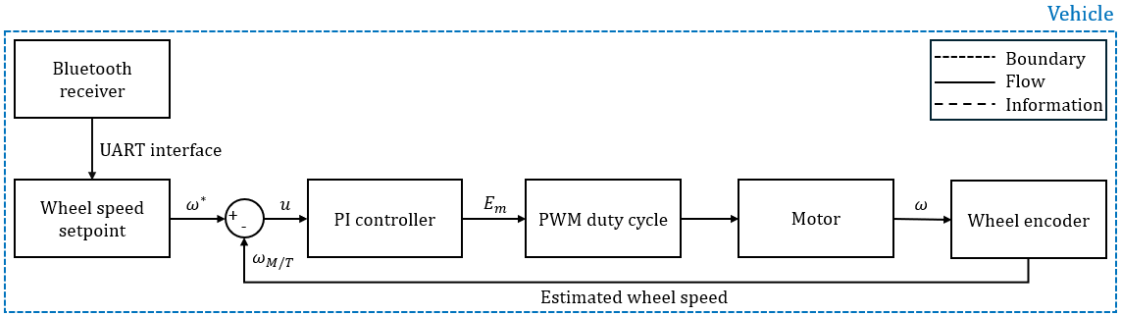


Figure 5.1.1: Program flow of the vehicle firmware

5.1.1 Communication

An HC-06 Bluetooth module on each vehicle (`VEH1` to `VEH3`) appears to the laptop as a virtual COM port. It uses the ATmega328P's hardware UART at 115 200 baud, because a software serial port blocks interrupts while each byte is transferred, which would delay the encoder timestamps. The same firmware runs over USB for bench testing, with the HC-06 disconnected. Commands take the form `S<wL>:<wR>`, setting both wheel-speed references in rad/s, and the vehicle answers each with a `.csv` telemetry row, `id, tMs, spL, wL, uL, spR, wR, uR`, giving its clock and each wheel's setpoint, measured speed and commanded voltage. The laptop logs these rows for every run.

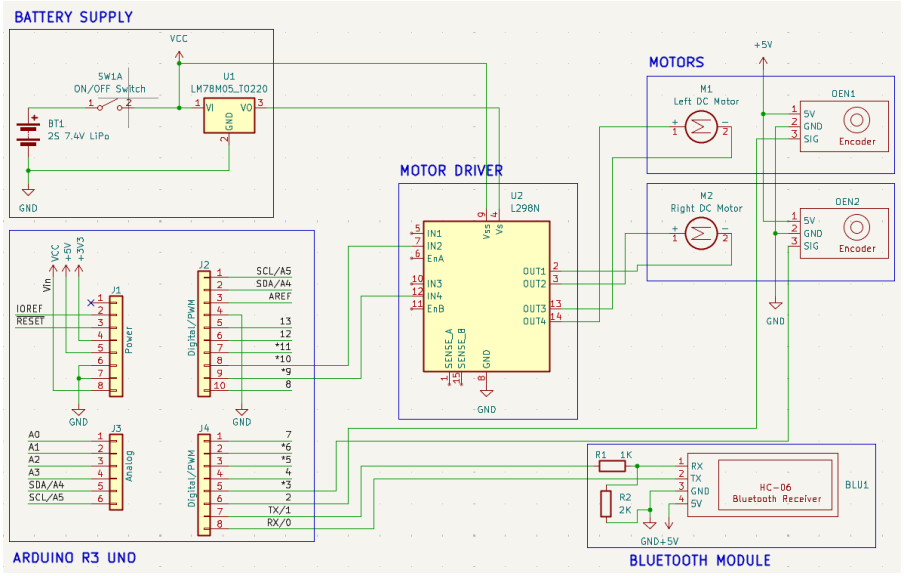


Figure 5.1.2: Schematic of vehicle hardware connections

5.1.2 Wheel speed estimation

Wheel speed is estimated by the M/T method. This provides better resolution at low speeds than M-method (encoder pulses, M_0, counting in a fixed window) and better high speed performance than T-method (counting Δt between pulses).

$$\omega_{M/T} = (\frac{f_0 M_0}{C M_1})2\pi, \tag{5.1.1}$$

where f_0 is the known high-frequency clock, M_0 is the encoder pulses in a fixed sample time,M_1 is the number of high-frequency pulses in the fixed sample time and C is number of encoder pulses per revolution.

The 20-slot discs are counted on both edges ($C = 40$) and timed with a 250 kHz clock. The wheels operate between 5 rad/s, a floor set by drivetrain stiction, and 10.4 rad/s, reached by the outer wheel when turning at $\omega_{\max}$. Above 2.62 rad/s every 60ms tick receives at least one new edge.

At the floor, the clock count gives a resolution of $\pm 6 \times 10^{-4}$ rad/s. The estimate is the mean speed over the span, so it lags the tick by at least $T/2 = 30$ ms. The tick itself is set by the encoder: a disc with more slots would allow a shorter tick and a faster speed loop but was not achievable within the budget limits and the accuracy was sufficient to allow for PI control at the desired set point.

5.1.3 Wheel speed control

Each wheel has a PI controller on the speed error $e = \omega^* - \omega$, with its output u in volts. It runs on the estimator's tick and is discretised with the Tustin transform

giving

$$i[k] = i[k-1] + \tfrac{1}{2}K_i T\big(e[k] + e[k-1]\big), \qquad u[k] = \mathrm{sat}_{[0,\,u_{\max}]}\big(K_p e[k] + i[k]\big). \quad (5.1.2)$$

The lower limit is zero because the vehicles only drive forward, and $u_{\max} = V_{\text{bat}} - V_{\text{drop}}$ is the largest voltage the driver can apply (Section 5.1.4). Conditional integration prevents windup: while the output is saturated, an increment is applied only if it moves the output back into range. Section 6 will cover the the tuning of the PI controller.

5.1.4 Motor drive

At the maximum output current, the bipolar output stage of the L298N driver drops $V_{\text{drop}} = 1.5$ V on the left channel and 1.8 V on the right, so the motor voltage is $u = DV_{\text{bat}} - V_{\text{drop}}$ for a duty cycle D. The firmware inverts this relationship to set the duty cycle. The drop is significant: at the battery's nominal 7.4 V, at most 5.6 V reaches the right motor.

Only `IN2` and `IN4` are switched, since the vehicles only drive forward. Setting the Timer 1 prescaler to one raises their PWM frequency from 490 Hz to 31.4 kHz. At 490 Hz the motor current became discontinuous, seen as ringing on the motor terminals; at 31.4 kHz it stays continuous, so the motor responds to the average voltage, as its model in Chapter 6 assumes.

5.2 Path Follower

The path follower runs on the laptop at the camera frame rate of 30 frames per second. For each frame it corrects lens distortion, estimates each vehicle's pose from its ArUco marker and computes wheel-speed references with a Pure Pursuit controller (Figure 4.3.1).

5.2.1 Calibration and pose estimation

Marker detection uses MATLAB's built-in ArUco detection library. The camera is calibrated with Zhang's method (Zhang, 2000) from 20 images of a 60mm checkerboard board, which gives the intrinsic matrix and distortion coefficients with an RMS reprojection error of 0.3885 pixels, and each frame is undistorted before detection. The markers come from the 4x4 dictionary. Its few bits give large cells that stay readable at the edge of the area, and because only seven identifiers are needed, a small dictionary keeps them far apart in Hamming distance, which reduces misidentification.

The four reference markers define a homography from the image to the floor, and each vehicle marker's corners are mapped through it to give the vehicle's position and heading. The vehicle markers sit $h_m = 49$ mm above the floor, so the homography places them too far from the point below the camera by a factor of

$H_c/(H_c - h_m)$, where H_c is the camera height; positions are scaled back by the inverse factor. The marker's offset from the axle centre (110 mm) is then removed using the heading. The capture buffer holds a single frame and the exposure is fixed, giving a latency from capture to command of 0.25 ms.

5.2.2 Pure Pursuit

Pure Pursuit (Coulter, 1992) steers each vehicle along the arc to a goal point on the path a look-ahead distance L_d ahead, commanding the turn rate $\omega = 2v \sin\alpha / L_d$, where α is the angle from the vehicle's heading to the goal point. The command is limited to $\pm\omega_{\max} = \pm 1.29$ rad/s, at which the inner wheel reaches its floor, and converted to wheel-speed references with the differential-drive inverse kinematics,

$$\omega^*_{L,R} = \frac{v \mp \omega L_{track}/2}{R}, \tag{5.2.1}$$

with $R = 32.5$ mm and $L_{track} = 136$ mm. The goal-point search starts from the previous frame's path segment, so the goal only moves forward, and the final waypoint becomes the goal once it is within L_d.

Linearised about a straight path, Pure Pursuit has a damping ratio of $1/\sqrt{2}$ for any speed and look-ahead, and a natural frequency of $\omega_n = \sqrt{2}\, v/L_d$. A short look-ahead tracks closely, while a long one responds more slowly and cuts corners more. Camera latency and the wheel-speed loops reduce the damping, so L_d is chosen to keep ω_n well below the wheel-speed loop bandwidth and the inverse of the camera latency, this parameters will be set in Chapter 8. Pure Pursuit corrects cross-track error but not timing along the path, so speed errors and corner cutting accumulate as along-track drift that the separation margin d_{min} must cover.

Chapter 6

Route planner selection

The route planning search model is a critical decision in this project as all other parts are aimed at testing if the produced paths can be practically followed and implemented. Thus to determine which is most effective at meeting all previous requirements, 3 solutions will be considered.

6.1 Considerations and limitations

It is assumed that the each vehicles' start and desired end locations are known at the start of the planning process. These start and end locations also only fall on the movement area boundary.
Secondly, the vehicles' speeds are the same and constant, meaning that each grid step takes a constant amount of time, and that is consistent for all vehicles.
Thirdly, computational time is assumed as negligible due to the complexity of these algorithms and that they will be running on a laptop which has sufficient processing power to plan the paths rapidly ($<$0.1s). This will be checked for the chosen route planner.

6.2 Prioritized A* Search

A decentralised basic A* search was applied due to its low computational time and ease of implementation. The implementation is decentralised as it makes use of a priority system. The first vehicle is confronted with no obstacles when planning, then the second has to plan with the first as a dynamic obstacle, and the third has to contend with both the first and second.
A* Search works by looking at which move (either left, forward or right) moved the vehicle closer to it's goal, but also moves it further away from it's start location. This is expressed by the cost equation 2.3.1, where $f(n) = g(n) + h(n)$. This is done as $g(n)$ represents the cost to the current node n and $h(n)$ is the estimated cost the the destination.
As a basis, it made use of a 3-direction system, so the vehicles could only move left, forward or right creating a 4-connected grid. This makes for a segmented path from the start-node to the finish-node, with 90° corners that a real-life vehicle will round when following. This is not a problem, it must just be accounted for in planning. The final solution in Section 5.3 will expand on the use of a well-clear distance to provide a buffer for this.

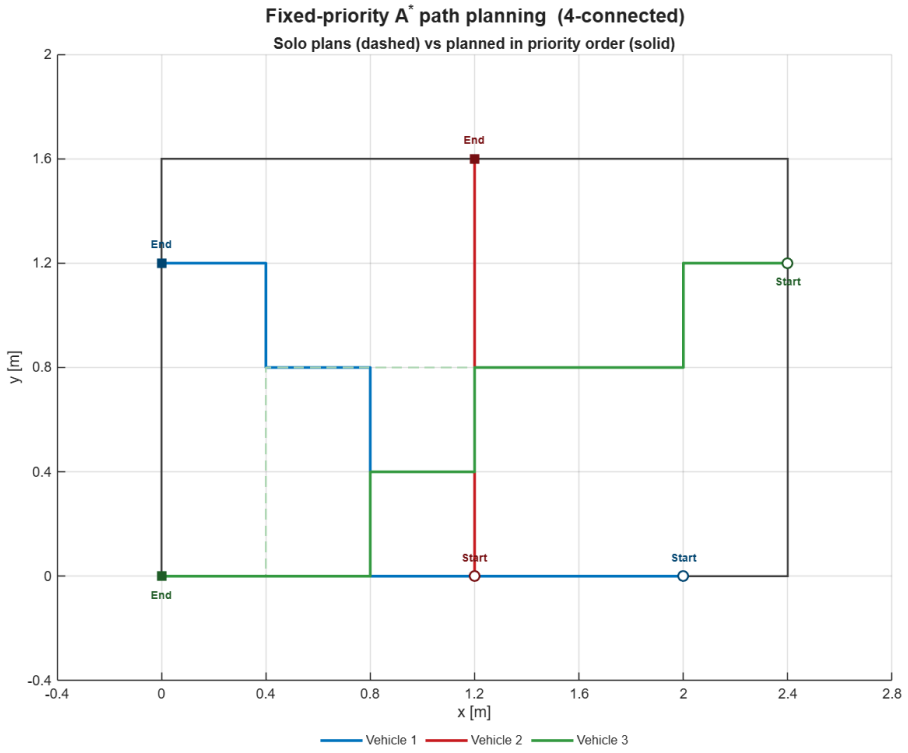


Figure 6.2.1: A* Search with 3 directions

6.3 A* Search with no priority

To improve planner efficiency and produce more cooperative paths, a brute force repetition approach was tried. All combinations of vehicle priorities were considered, not limiting vehicle 1 to have the highest priority, with the objective of determining the shortest total travel distance by the vehicles within the movement area. By doing so it runs 9 iterations of the previous route planner except on an 8-connected grid so that the vehicles can now go straight, turn 45°, and turn 90°.

This implementation shows that with the adoption of of well-clear distance, this could be a viable model. Though doesn't produce the shortest paths and if high resolution is required, may require non-negligible computation time.

6.4 Collaborative A* Search

Both models presented so far plan one vehicle at a time. The first fixes the priority order in advance; the second searches over different sequences to find the fairest

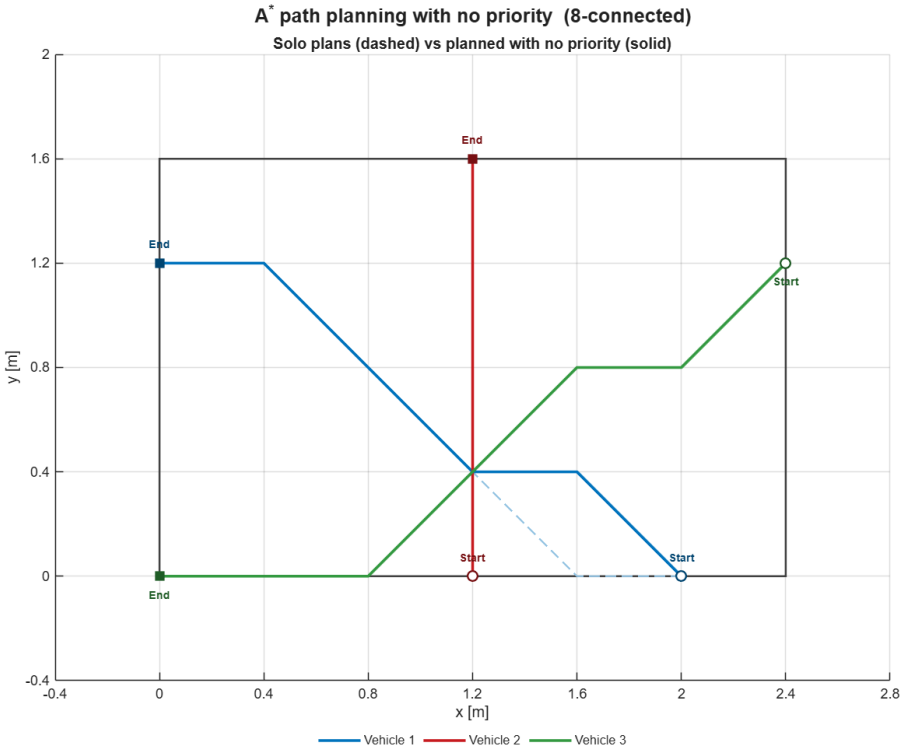


Figure 6.3.1: A* Search with 5 directions and no priority

one. Neither is collaborative during planning. In both, the second and third vehicle is routed against a set of already-committed paths it must treat as immovable, and neither can guarantee that the lowest-cost set of paths is ever found. To correct this, a collaborative A* search was built, in which every vehicle is planned simultaneously.

The search operates on the joint state of the whole fleet. A joint state records, for each vehicle its pose (cell location and heading) and the number of steps it has used so far. At every timestep each vehicle independently chooses to move left, straight or right, so with three vehicles there are $3^3 = 27$ distinct joint decisions per timestep. Because no vehicle is planned first, no vehicle is prioritized: the search trades all three routes off against one another at once and returns the globally best joint plan within its search bounds, which a priority-based planner cannot do. This poses a much more complex computational task though creates better results. The adjustments made to improve both planner and computational perfromance are explained below.

6.4.1 Heuristic adjustments

The standard A* Search heuristic (Equation 2.3.1) is admissible on a 4-connected grid, but fails to account for heading.

It is desired for vehicles to leave the movement area perpendicularly, as a vehicle leaving in-line with the border could create a collision with another vehicle leaving the boundary at another point which the planner could not solve for. A vehicle leaving perpendicularly solves this problem.

Manhattan distance cannot deal with this scenario as a vehicle at it's end location, regardless of heading will result in $h(n) = 0$. Since the vehicles cannot move backwards, this could actually mean that as many as 4 moves are still required of the vehicle to exit with the correct heading.

The heuristic was therefore replaced with an exact distance-to-goal field. For each vehicle v, a reverse breadth-first search (BFS) is run once over the (cell, heading) graph, seeded at the goal state (i_g, j_g, θ_g) and expanding backwards along reversed moves. This yields a table

$$h_v(i, j, \theta) = \text{exact number of moves from } (i, j, \theta) \text{ to the goal, arriving perpendicular,} \tag{6.4.1}$$

which is the true single-vehicle optimal cost, including every turn the limited motion forces and the correct exit heading. The field is computed once per vehicle and read by table lookup thereafter, so the tighter bound costs essentially nothing at search time.

Two things follow from having the exact field rather than an approximation. The joint heuristic is simply the sum of the per-vehicle values, and the solo-optimal cost of each vehicle is available for free as $d_v = h_v(\text{start})$ - which is what makes the pruning of Section 6.4.4 possible.

6.4.2 Cost function

The objective is sum-of-costs: minimise the total number of grid steps summed over all vehicles. For a joint node n holding N vehicles with states $s_v = (i_v, j_v, \theta_v)$,

$$f(n) = g(n) + h(n) = \sum_{v=1}^{N} g_v(n) \; + \; \sum_{v=1}^{N} h_v(i_v, j_v, \theta_v), \tag{6.4.2}$$

where $g_v(n)$ is the number of steps vehicle v has taken to reach the node and h_v is the field of Equation 6.4.1. A joint node in which all three vehicles are still driving therefore accrues a cost of 3 per timestep; once a vehicle exits, subsequent timesteps cost 2, then 1.

It is worth being explicit about what this objective does and does not optimise. This aims to create the fairest plan for the fleet of vehicles even if it causes greater detours for 1 vehicle.

For a given scenario, if it results in the total lowest fleet time to put all the deviation on one vehicle, the route planner does so, if it is lower to deviate 2 or 3 vehicles slightly, it will output that plan instead.

6.4.3 Well-clear distance

Treating a vehicle as a dimensionless point that merely has to avoid occupying another vehicle's cell is not sufficient. The vehicles have physical dimensions to them, and more importantly the route actually driven is not the route that was planned due to vehicles dynamic constraints, sensor noise and the nature of waypoint following controllers.

Each vehicle v carries a protected disc of radius r_v, and a pair (a, b) is in conflict whenever those discs overlap:

$$\|\mathbf{p}_a(t) - \mathbf{p}_b(t)\| \;<\; R_{ab}, \qquad R_{ab} = r_a + r_b. \tag{6.4.3}$$

Imposing this only at timesteps is not sufficient, because two vehicles can occupy cells at both ends of a step and still pass too close part-way through it. Thus the minimum distance between vehicles between cells needs to be determined and can be done so analytically as the vehicles move at constant speed in a straight line over a fixed distance between cells. Thus, the minimum separation between two vehicles can be determined by:

$$\tau^* = \operatorname{clamp}\!\left(-\frac{\mathbf{r}_0 \cdot \Delta\mathbf{r}}{\|\Delta\mathbf{r}\|^2},\ 0,\ 1\right), \qquad d_{\min} = \left\|\mathbf{r}_0 + \tau^*\Delta\mathbf{r}\right\|, \tag{6.4.4}$$

where $\mathbf{r}_0$ and $\mathbf{r}_1$ are the relative displacements at the start and end of the step and $\Delta\mathbf{r} = \mathbf{r}_1 - \mathbf{r}_0$. Appendix A derives this.

Because the minimum is obtained in closed form rather than by sampling, no near-miss can fall between evaluation points. The test overrides the cell rules it replaces: two vehicles sharing a cell give $d_{\min} = 0$, and a head-on exchange of cells passes through $d_{\min} = 0$ half way through the step making for a more efficient detection method.

Two consequences are specific to enforcing a well-clear distance on a discrete grid. Firstly, the grid can only place vehicles at whole-cell spacings, so raising the radii does not degrade efficiently. Any pair whose radii sum to more than one cell size can never occupy 4-neighbour cells at all, reducing possible paths and increasing likelihood of a scenario in which a path in impossible to find. Secondly, as the vehicles cannot follow the paths with 90° corners and will instead drive a radius, the vehicles will deviate from their paths both in cross-track distance/error (XTE) and take less time if on the inside of the corner or more time if on the outside of the corner to follow the path, thus deviating in in-track distance/error (ITE). This means that the well-clear distance needs to be large enough to ensure

that even when a vehicle deviates from the path in time and position, the vehicles still do not collide. There is variability in both XTE and ITE, and the well-clear distance needs to be able to ensure no collisions occur between the physical vehicles in these scenarios.

6.4.4 Time and computation optimisation

A simple 27-way joint search with no optimisation is intractable. The joint state space here is (cells $\times$ 4 headings)^3, of the order of 10^8 states. Four mechanisms reduce this, in increasing order of importance.

Exact heuristic - The per-vehicle distance field of Equation 6.4.1 is a tighter admissible bound than Manhattan distance, which directly reduces the number of nodes A* must expand.

Detour budget - This is the mechanism that narrows the search as more of the route becomes determined. Each vehicle's solo-optimal cost $d_v = h_v(\text{start})$ is a hard lower bound on what it can ever achieve. Each vehicle is allowed a slack budget B, and any branch in which

$$g_v + h_v \; > \; d_v + B \tag{6.4.5}$$

is discarded immediately. Early in the search g_v is small and almost every action passes; as a vehicle commits more of its route g_v grows and its remaining slack shrinks, so the set of admissible actions collapses towards the end of the search. It can only exclude plans in which some vehicle detours by more than B steps - and if no plan exists within B, the budget is increased automatically and the search retried.

Operator decomposition - Rather than generating all 27 successors of a node at once, the vehicles are moved one at a time through intermediate states, so the branching factor is 3 rather than 27. A* can then discard a hopeless vehicle moves that result in the vehicle moving outside the movement area or entering a location the goal cannot be reached from, before ever considering what vehicles 2 and 3 would do beneath it. The set of joint plans searched and the plan returned are identical; only the wasted expansions differ. Measured on this scenario it generates about 2.5$\times$ fewer nodes with three vehicles.

Well-clear pre-filter - Joint actions containing a well-clear separation violation are pruned as they are generated using Equation 6.4.4.

Altogether, these reduce the centralized search to 0.09 s on the scenario shown and returns a plan that no prioritised method can reach. This outputs a '.csv' file of the x and y-cell coordinates as waypoints to that will be used as the input to the path follower programme.

The result of this route planner can be seen in Figure 6.4.1.

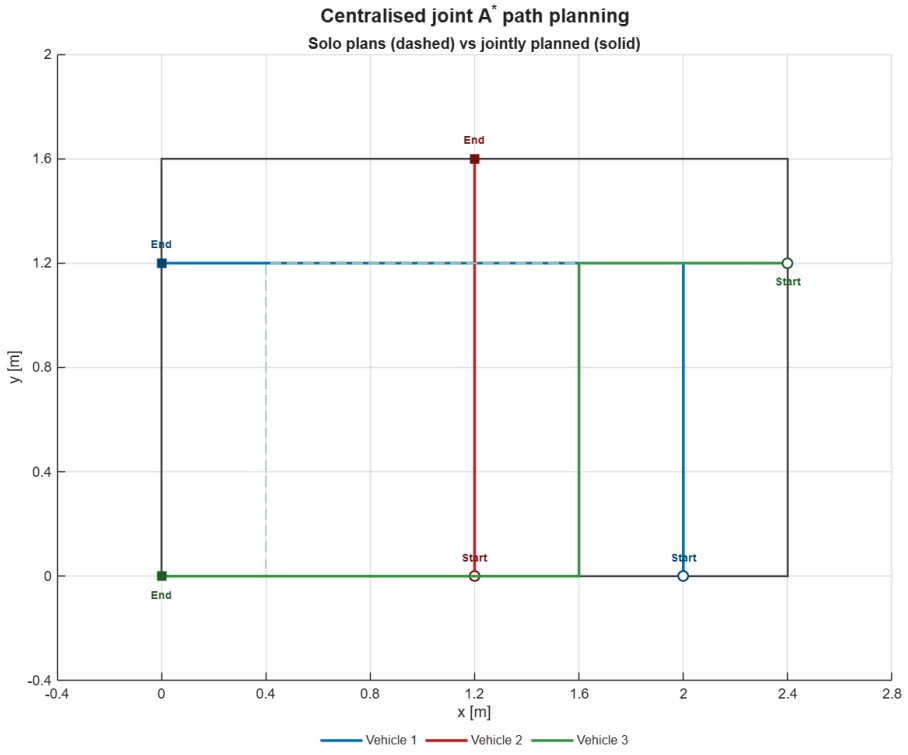


Figure 6.4.1: Collaborative (centralised joint) A* Search, planning all three vehicles simultaneously on a 4-connected grid

Chapter 7

Vehicle modelling

Each wheel's speed controller needs a model of its drivetrain. This chapter identifies the motor parameters, fits a vehicle model, compares the three vehicles and tunes the controllers.

7.1 Motor Model

The motors have no published parameters, so they are identified by test. Each motor obeys

$$E = R_a I + L_a \dot{I} + K\omega_m, \qquad T_m = KI, \tag{7.1.1}$$

where E is the applied voltage, I the armature current, Ω_m the motor speed, T_m the motor torque, R_a and L_a the armature resistance and inductance, and K the motor constant. The inductance is neglected to simplify the model.

7.1.1 Armature resistance

With the rotor locked and the current steady, $\omega_m = \dot{I} = 0$ and Equation (7.1.1) reduces to $E = R_a I$. A bench supply was swept from 2 to 6 V, and R_a is the slope of the fitted voltage–current line (Figure 7.1.1), which keeps any brush drop in the intercept. Across the six motors (averaged), $R_a = 8.71\ \Omega$.

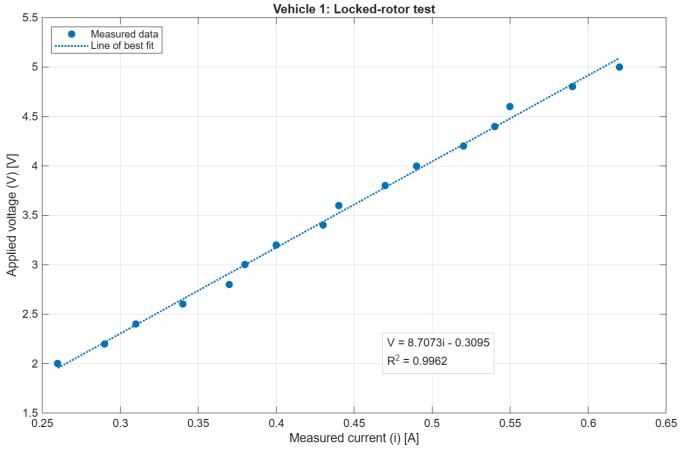


Figure 7.1.1: Locked-rotor test, vehicle 1 left motor

7.1.2 Motor constant

With the wheel removed and the vehicle off the floor, the supply was stepped from 3 to 6 V in 0.2 V increments, recording the steady-state current and output-shaft speed Ω. With the gear ratio $N = 48$,

$$K = \frac{E - R_a I}{N\Omega} \tag{7.1.2}$$

averages 2.81 mV s/rad. Gearbox friction raises the current but does not bias K. Only vehicle 1's motors, the most stable of the six (Section 7.3), were tested, so K is assumed common to all motors.

7.2 Vehicle Model

From the free-body diagrams in Figure 7.2.1, derived in Appendix B, the straight-line response ($\omega = 0$) of each wheel to its motor voltage is

$$\frac{\Omega_{L,R}(s)}{E_{L,R}(s)} = \frac{2NK}{m_{\text{eff}} r_w^2 R_a\, s + b_{\text{eff}} r_w^2 R_a + 2N^2K^2} = \frac{G}{\tau s + 1}, \tag{7.2.1}$$

with vehicle speed $v_C = R\omega_{L,R}$. The effective mass $m_{eff} = m + 2(J_w + N^2 J_m)/r_w^2$ adds the reflected wheel and rotor inertia to the vehicle mass, and b_{eff} lumps rolling, gearbox and bearing losses into one viscous term.

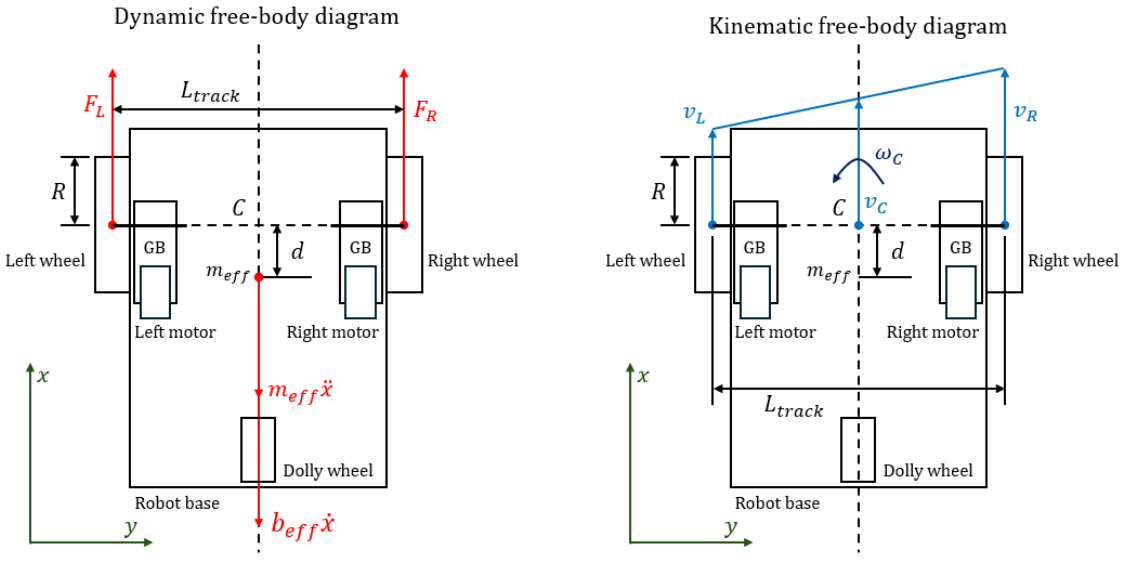


Figure 7.2.1: Dynamic and kinematic free-body diagrams of the vehicle

7.2.1 Parameter fitting

The effective mass and damping were fitted to open-loop voltage steps: b_{eff} sets the steady-state speed and m_{eff} the rise time. The fit gives $m_{eff} = 0.85$ kg and $b_{eff} = 4.5$ N s/m. Against a vehicle mass of 470 g, reflected inertia makes up 44.7 % of m_{eff}. Figure 7.2.2 shows the fit for vehicle 1. The left wheel runs faster because the driver drops less voltage on that channel (Section 5.1.4). Coulomb friction, which sets the 5 rad/s floor, is absorbed into b_{eff}, so the model holds only from 3 to 6 V.

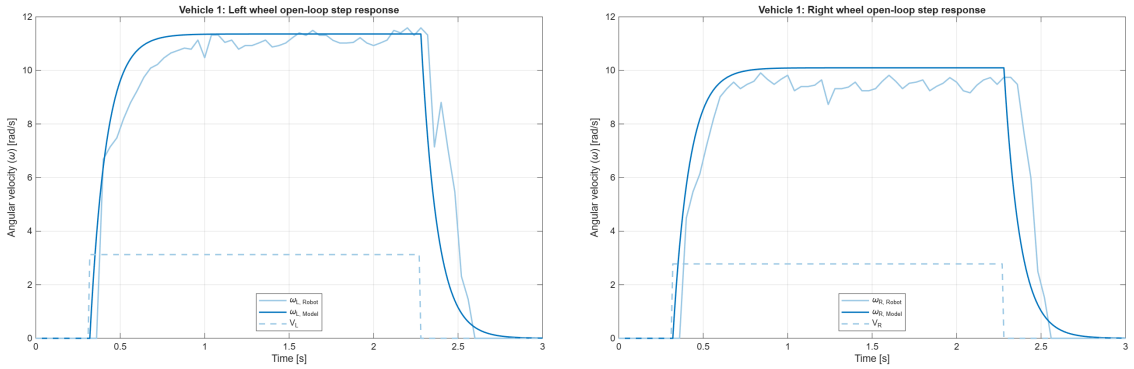


Figure 7.2.2: Vehicle 1, equal voltages on both wheels, with model

7.2.2 Turning

Rather than identifying the yaw inertia and turning friction, each wheel is modelled independently by Equation (7.2.1), with the vehicle's motion following from the kinematics. This is exact in a straight line and approximate in turns, where the chassis couples the wheels through its yaw inertia. The error is small because reflected inertia, which acts on each wheel alone, dominates m_{eff}, and because the wheels stay within ±35 % of the straight-line 7.69 rad/s. Figure 7.2.3 confirms this with 5 V on the left wheel and 4 V on the right.

7.3 Fleet Variation

Vehicle 1's wheels hold steady state with a ripple of about 0.5 rad/s, some of which may be encoder quantisation. Vehicles 2 and 3 rise like the model but then oscillate by 2 rad/s or more, and vehicle 3 settles below the model (Figures 7.3.1 and 7.3.2). Swapping motors between vehicles moved the oscillations with the motors, ruling out the vehicles' electronics and software.

The oscillations occur at 10-12 Hz at a wheel speed of 11.5 rad/s, an order of $n = f_{osc}/f_{wheel} \approx 6$ /wheel revolution or every 60°. This rules it as motor cogging. This makes sense for DC motors with 3-pole pairs. It is caused by the permanent

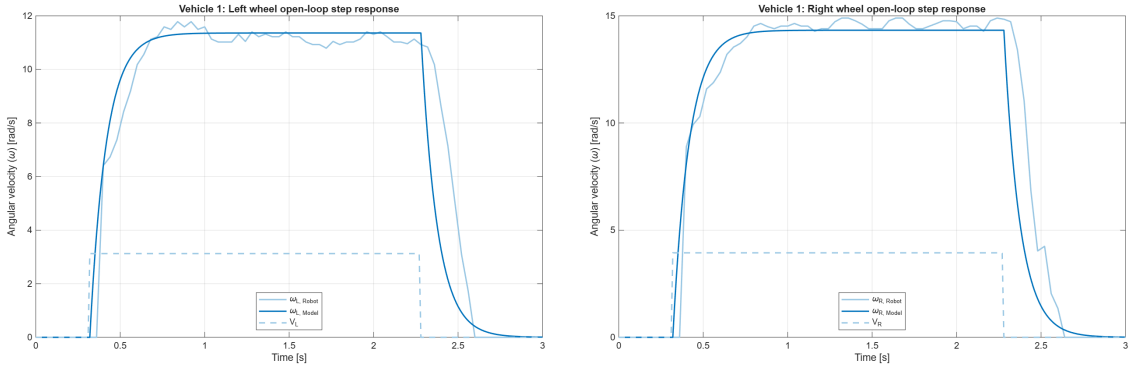


Figure 7.2.3: Vehicle 1, 5 V left and 4 V right, with model

magnets in the motors being attracted to the iron teeth of the armature laminations. This becomes prevalent when driving motors at low speeds. Gearbox lubrication will not meaningfully improve performance as it is a function of the motors design. Motors with more poles or ironless cores would need to be purchased to solve this problem.

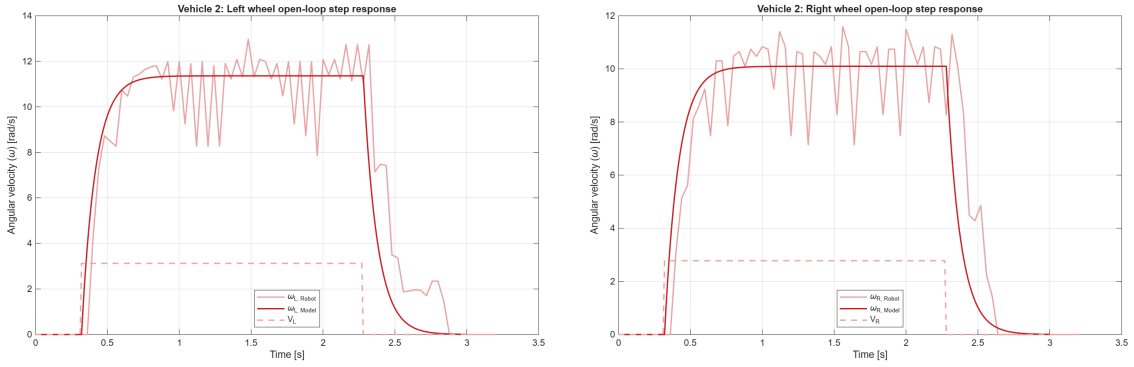


Figure 7.3.1: Vehicle 2, equal voltages on both wheels, with vehicle 1 model

Because vehicles 2 and 3 provide such variable data, the tuned values from vehicle 1 will be used in all simulations.

7.4 Speed Controller Tuning

The PI controllers (Section 5.1.3) were tuned on vehicle 1 for zero steady-state error and to overcome stiction on the first control tick From rest, the first output for a step to ω^* is $u[0] = (K_p + K_i T/2)\,\omega^*$. With the tuned values of $K_p = 0.18$ V s/rad and $K_i = 2.5$ V/rad, the PI zero is at $K_i/K_p = 13.9$ rad/s. It lies near the plant

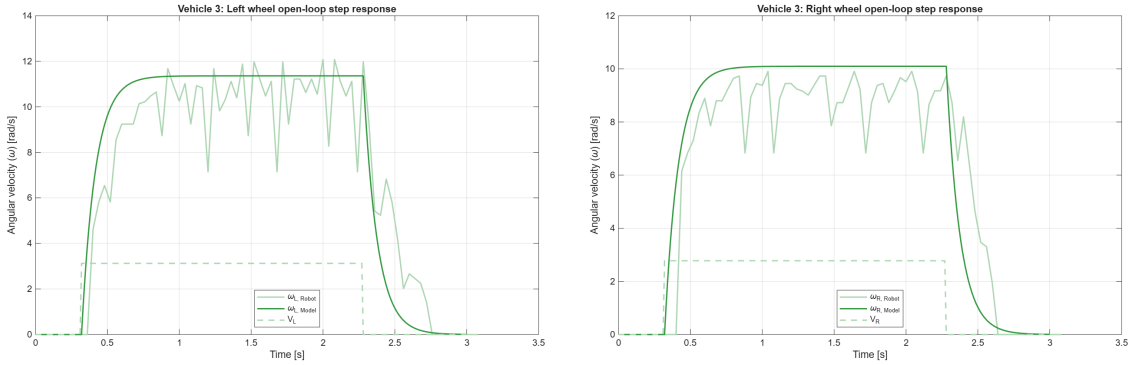


Figure 7.3.2: Vehicle 3, equal voltages on both wheels, with vehicle 1 model

pole at $1/\tau = 10$ rad/s, and the loop crosses over at 7.8 rad/s (1.25 Hz) with a phase margin of about 50°, allowing for about 70 ms of hold and estimator delay.

Figure 7.4.1 compares vehicle 1's closed-loop response with the model, which runs the same M/T estimator as the firmware. The estimator matters at start-up: its output stays at zero until the first full edge interval, and the integrator accumulates the full error in the meantime. The measured response rises ≈ 0.3 s, overshoots by 10 % and matches the model to within 0.2 rad/s RMS. The model matches equally well in a turn (Figure 7.4.2), so the closed-loop model can be used to simulate path following.

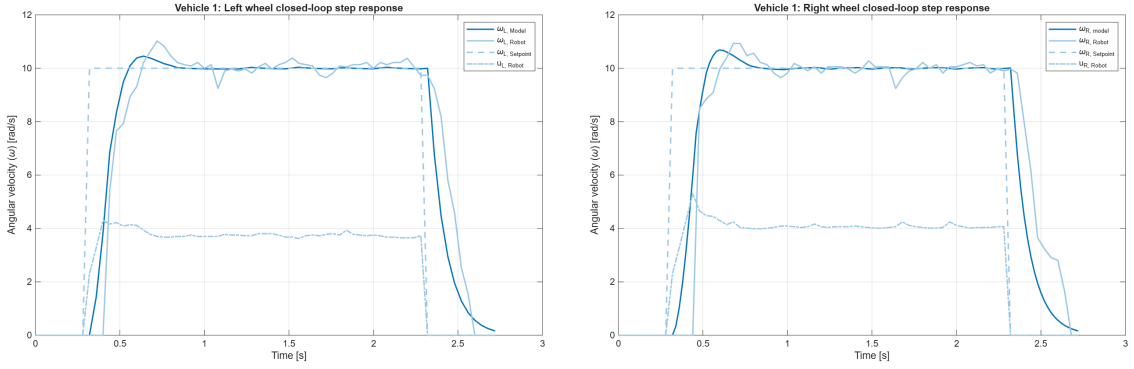


Figure 7.4.1: Vehicle 1, straight-line closed-loop step response, with model

On vehicles 2 and 3, with the same gains, the six-per-revolution oscillation persists in closed loop with an amplitude of TBD rad/s (Figures 7.4.3 and 7.4.4). The controller cannot reject it: at 7.69 rad/s the oscillation is at 7.3 Hz, far above the 1.25 Hz crossover, where the loop has almost no effect. The ripple barely affects path following, because a wheel-speed ripple of amplitude A at frequency f

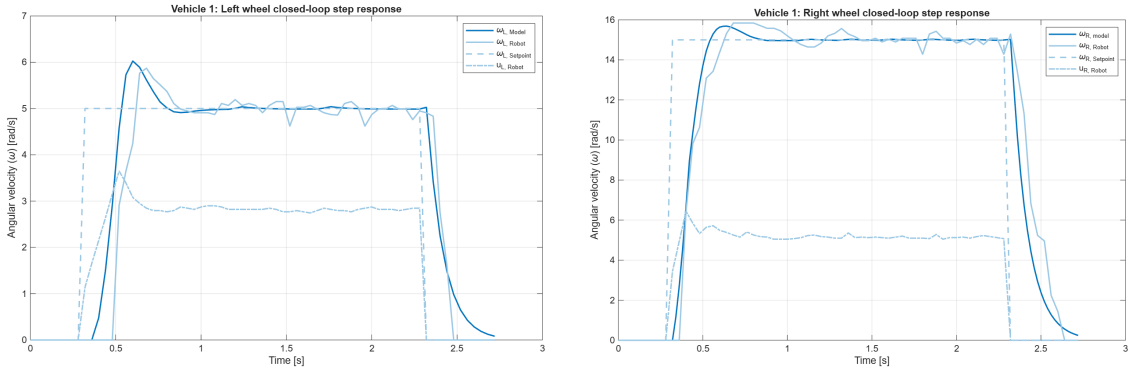


Figure 7.4.2: Vehicle 1, turning closed-loop step response, with model

displaces the vehicle by only $r_w A/(2\pi f)$, about 1.4 mm for 2 rad/s at 7.3 Hz. Path following depends instead on the mean speed.

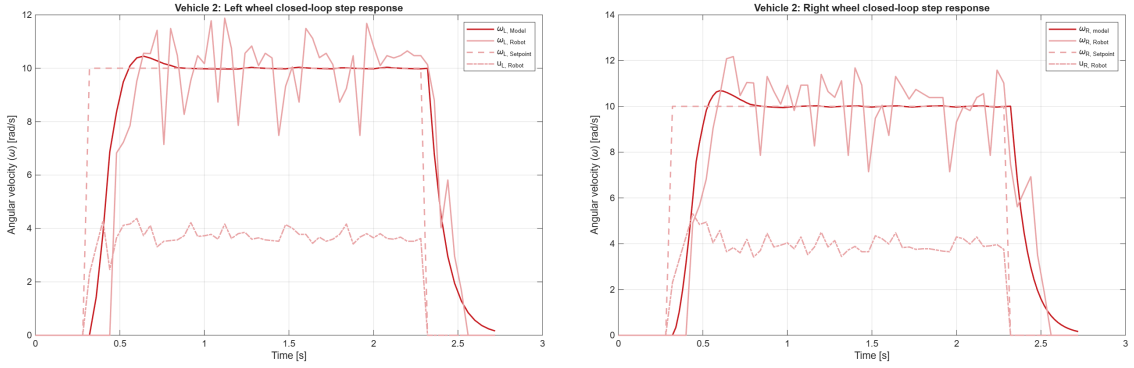


Figure 7.4.3: Vehicle 2, straight-line closed-loop step response

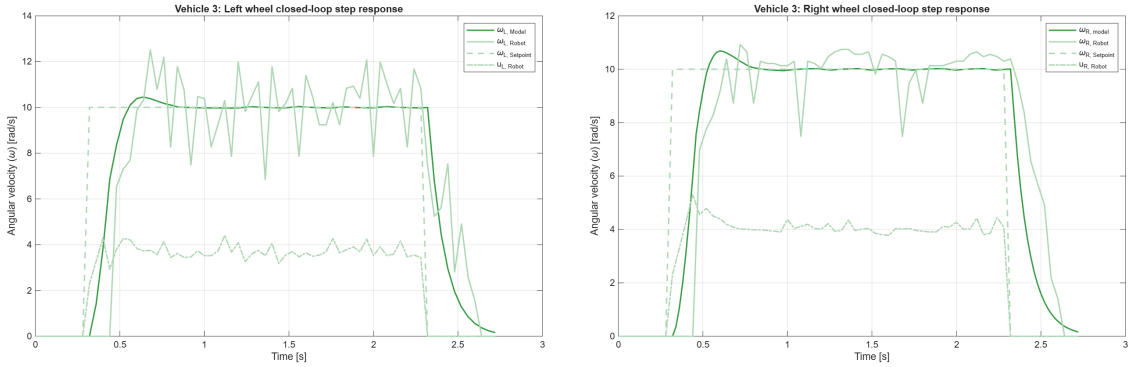


Figure 7.4.4: Vehicle 3, straight-line closed-loop step response

Chapter 8
Simulation

The Simulink model has three nested levels that mirror the hardware: a wheel-speed loop for each side of a vehicle, a vehicle model that combines two wheel loops with the kinematics and the camera measurement, and a path-following loop that drives one vehicle along its planned path. The path-following loop is repeated for each vehicle. The model uses a fixed-step solver with a step of 20 ms in single-tasking mode, so blocks execute in the same order as the firmware.

8.1 Wheel-speed loop

Figure 8.1.1 shows one side of a vehicle, which reproduces the wheel-speed loop of Section 5.1.3. The discrete PI controller acts on the error between the reference ω^* and the M/T estimate at the 60 ms tick. Its output is rounded to the resolution of the 8-bit PWM, and the driver block applies the characteristic of Section 5.1.4 to give the voltage at the motor. The motor is the first-order model of that wheel. Its output, the true wheel speed, feeds the M/T estimator, which runs the same code as the firmware on the simulated wheel angle, and the estimate closes the loop. The block outputs both the true and the estimated speed to allow for the vehicle speed to be integrated to determine it's pose.

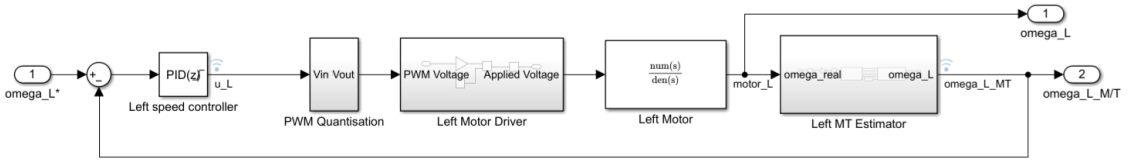


Figure 8.1.1: Wheel-speed loop for one side of a vehicle (left side shown)

8.2 Vehicle

Figure 8.2.1 shows the full vehicle. The commanded speed v_V and turn rate ω_C are converted into wheel-speed references with the inverse kinematics of Equation (5.2.1) and delayed by the measured Bluetooth communication latency of

$\approx$ 200 ms. Each reference drives one wheel-speed loop. The forward kinematics then convert the true wheel speeds into the vehicle's speed and turn rate,

$$v_C = \frac{r_w\,(\omega_R + \omega_L)}{2}, \qquad \omega_C = \frac{r_w\,(\omega_R - \omega_L)}{b}, \tag{8.2.1}$$

and the pose rates $(\dot{x}, \dot{y}, \dot{\psi}) = (v\cos\psi,\ v\sin\psi,\ \omega)$ are integrated to give the true pose. The ArUco block turns the true pose into the measured pose that the path follower receives. It samples the pose at the camera frame rate, adds random position and heading errors matching the measured tracking accuracy, and delays the result by the measured pipeline latency of 250 ms.

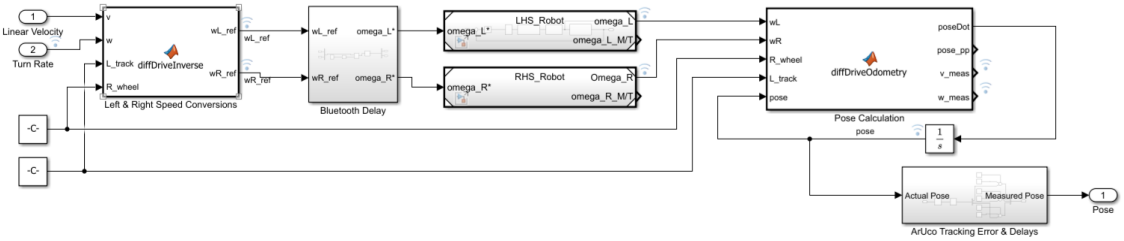


Figure 8.2.1: Vehicle model, including the ArUco tracking error and delays

8.3 Path-following loop

Figure 8.3.1 shows the path-following loop for one vehicle. The Pure Pursuit block (Section 5.2.2) takes the vehicle's waypoints from the planner and its measured pose, and outputs the linear speed and the path curvature, whose product is the commanded turn rate $\omega_C = v\kappa$. The goal block passes these commands to the vehicle until it is within 200 mm of its goal, then sets both to zero and flags the vehicle as finished. This ensures that the vehicles do not circle the destination creating spurious cross track error data. Each vehicle has its own copy of this loop, and the simulation stops once every vehicle has reached its goal. The logged true poses give each vehicle's tracking error and the separation between vehicles (Chapter 9).

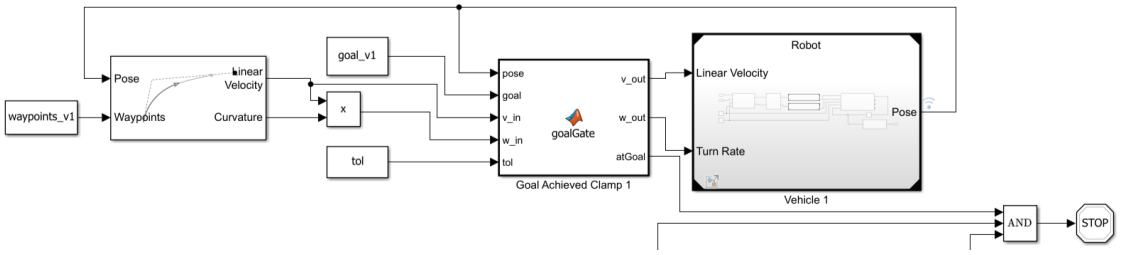


Figure 8.3.1: Path-following loop for vehicle 1

Chapter 9

Parameter setting

Three parameters link the planner to the vehicles: the grid size, the look-ahead distance and the well-clear distance. They are set in that order, because each depends on the one before.

9.1 Grid size

The grid must be coarse enough for the vehicles to follow any path the planner produces. The turn rate is limited by the inner wheel reaching its 5 rad/s floor, $\omega_{\max} = (v - r_w\omega_{\min})/(b/2)$, so at $v = 0.25$ m/s the minimum turn radius is

$$r_{turn,min} = \frac{v_C}{\omega_{C,max}} = \frac{L_{track}}{2}\,\frac{v}{v - R\omega_{min}} = 68\ mm \times \frac{0.25}{0.25 - 0.1625} = 194\ mm. \tag{9.1.1}$$

The tightest manoeuvres on a grid of spacing g are two 90° turns at adjacent nodes creating U-turn into the next lane. A 90° arc tangent to both grid lines starts $r_{\min}$ before its node and ends $r_{\min}$ after it, so two such arcs fit between adjacent nodes only if

$$g \geq 2r_{\min} = 389 \text{ mm}. \tag{9.1.2}$$

The grid size is therefore set to $g = 400$ mm.

9.2 Look-ahead distance

Without delays, Pure Pursuit has a damping ratio of $1/\sqrt{2}$ for any look-ahead. Delays however reduce it. A total delay τ_d from the camera, the Bluetooth link and the wheel-speed loops costs a phase of $\omega_n\tau_d = \sqrt{2}\,v\tau_d/L_d$ at the natural frequency, so the longest look-ahead keeps the most damping.

Cornering limits the look-ahead. A goal point more than one grid spacing away can lie beyond the next node, so the vehicle would skip a segment and cut the corner entirely. The look-ahead is therefore set to the largest value that still follows every segment, $L_d = g = 400$ mm, which gives $\omega_n = 0.88$ rad/s at 0.25 m/s.

9.3 Well-clear distance

Figure 9.3.1 shows the two distances around each vehicle. The collision boundary, of radius 200 mm, encloses the vehicle and its marker. The planner keeps each

vehicle's well-clear circle, of radius L_{WC}, from overlapping any other's, so the space between the two circles is a buffer for that vehicle's deviation from its plan. The buffer must exceed the largest position error a vehicle can make while following a planned path.

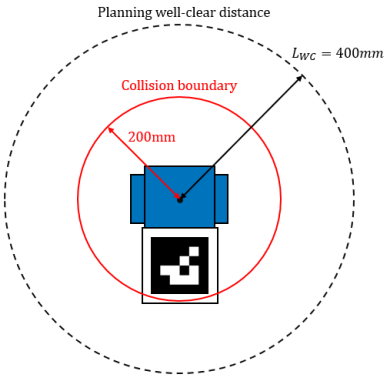


Figure 9.3.1: Collision boundary and planning well-clear distance around a vehicle

The square test in Figure 9.3.2 gives that worst case. A square of side g is the tightest closed path the grid allows, with a 90° turn every 400 mm, and with $L_d = 400$ mm the vehicle never settles on an edge. At $T = 6.40$ s, when a perfectly tracking vehicle would close the loop, the vehicle has driven only 0.96 of the perimeter and is 92 mm from the corner, its in-track error. Its peak cross-track error is 90 mm, beyond the top edge. The in-track and cross-track errors are perpendicular components of the position error $\mathbf{e}$, so

$$\|\mathbf{e}\| \leq |e_{\parallel}| + |e_{\perp}| \leq 92 + 90 = 182 \text{ mm}, \tag{9.3.1}$$

and the buffer must be at least 182 mm. Setting $L_{WC} = 400$ mm gives a buffer of 200 mm, a margin of 18 mm. Because each vehicle carries its own buffer, two collision boundaries stay apart even if both vehicles deviate towards each other by the full 182 mm.

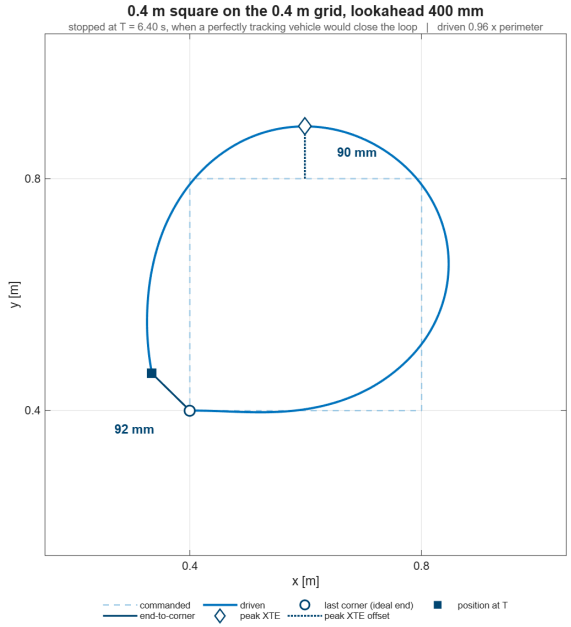


Figure 9.3.2: Square test on the 400 mm grid with a 400 mm look-ahead

Chapter 10
Monte-Carlo Testing

Testing every scenario on the hardware is infeasible, so the system was evaluated in simulation over 500 random three-vehicle scenarios, using the model of Chapter 7.

10.1 Setup

Each scenario places three vehicles at random start nodes on the boundary of the movement area, with random headings, and gives each a random destination node on the boundary. A scenario is accepted only if:

- Straight-line paths from start to destination would make at least one pair of vehicles collide, so that every scenario requires avoidance.
- The vehicles start at least L_{WC} apart.
- Each vehicle's start and destination lie on different edges, so that it crosses the area.
- Each vehicle starts heading into the area or along its edge.

No combination of starts and destinations appears twice. Each run is classified by the minimum centre-to-centre separation between any two vehicles: a collision below 200 mm, a near miss from 200 to 250 mm, below the planning requirement from 250 to 400 mm, and clean at 400 mm or more.

10.2 Success rate

Figure 10.2.1 summarises the outcomes, and Figure 10.2.2 shows the distribution of minimum separations. Five runs collided, a rate of 1.0 % with a 95 % Wilson interval of 0.4 to 2.3 %. The 99.0 % collision-free rate meets the criterion of at least 99 % only at the point estimate; because the interval extends to 2.3 %, the simulation cannot show that the criterion is met. Only 169 runs (33.8 %) kept every pair at least 400 mm apart, well short of the 85 % criterion. Of the rest, 314 runs dipped below the planning requirement without coming within 250 mm, and 12 were near misses.

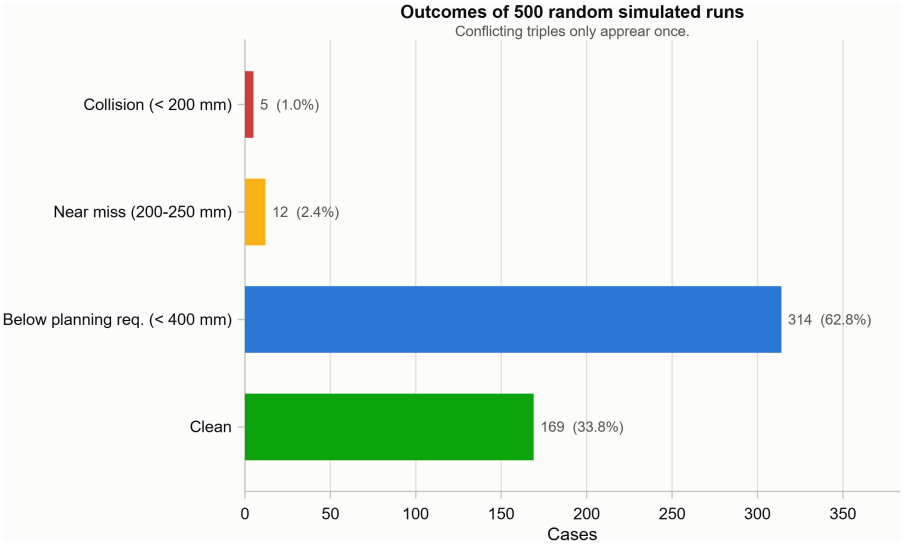


Figure 10.2.1: Outcomes of the 500 Monte-Carlo runs

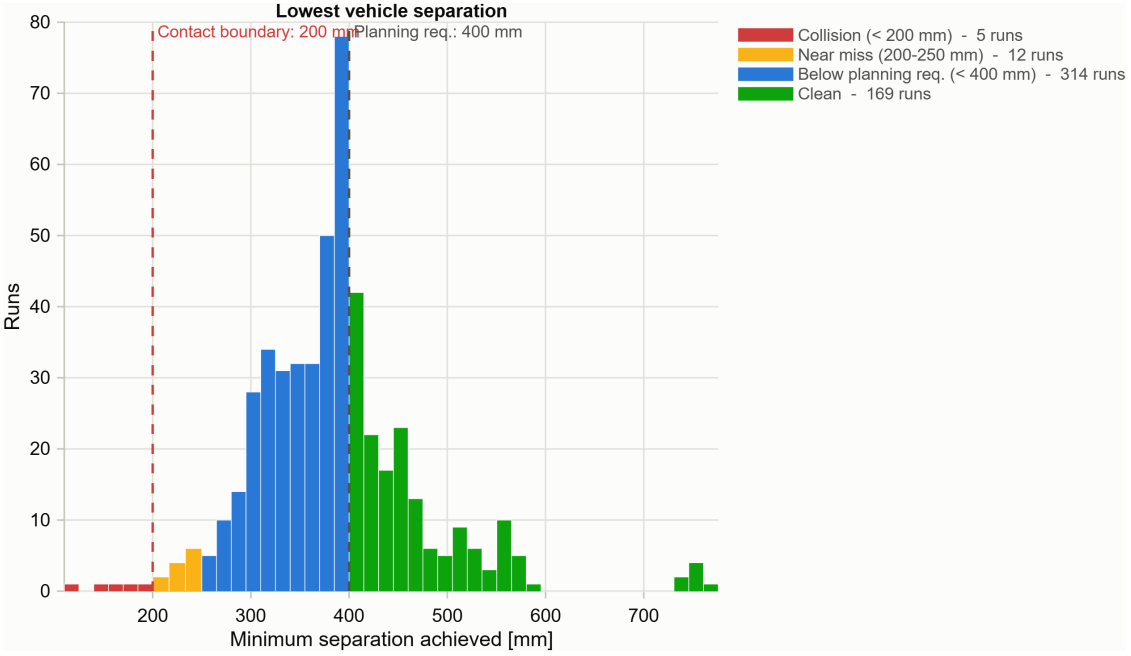


Figure 10.2.2: Minimum separation achieved in each run

10.3 Minimum separation

Figure 10.3.1 separates the runs by their planned minimum separation. In 458 of the 500 plans (92 %), the closest approach sits exactly at the 400 mm requirement, because the planner minimises path length and the separation constraint is active at the optimum. Tracking error then spreads the delivered separation around the plan, with a median of 381 mm. The 42 plans with margin above the requirement delivered a median of 547 mm and none fell below 400 mm, so all 331 runs below the requirement came from plans at it: 72 % of those plans.

The shortfall therefore comes from planning without a margin for tracking error, not from the planner itself. Planning to a separation of $L_{WC} + \delta$, where δ is the separation lost to tracking in 85 % of runs, would meet the criterion; Figure 10.3.1 puts δ at about 100 mm. A larger movement area helps only by keeping such plans feasible: without the margin, the planner would still place each encounter at exactly 400 mm.

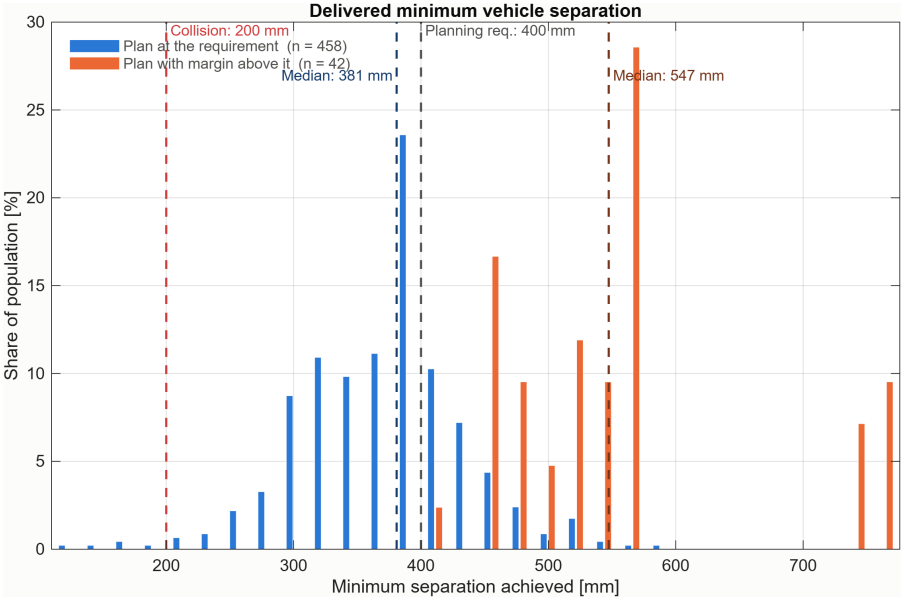


Figure 10.3.1: Delivered minimum separation for plans at the requirement and plans with margin above it

10.4 Cross-track error

Figure 10.4.1 shows the peak cross-track error of each of the 1500 vehicle-runs. The median is 96 mm, the 95th percentile 142 mm and the largest 178 mm. The

runs near zero are vehicles with straight paths; the clusters at 65–80 mm and 110–115 mm correspond to runs with only one corner. The square test of Section 9.3 predicted a peak of 90 mm, but more than half the vehicle-runs exceed it and the largest is nearly twice as large. The square is therefore not the worst-case path, and the buffer derived from it underestimates the error.

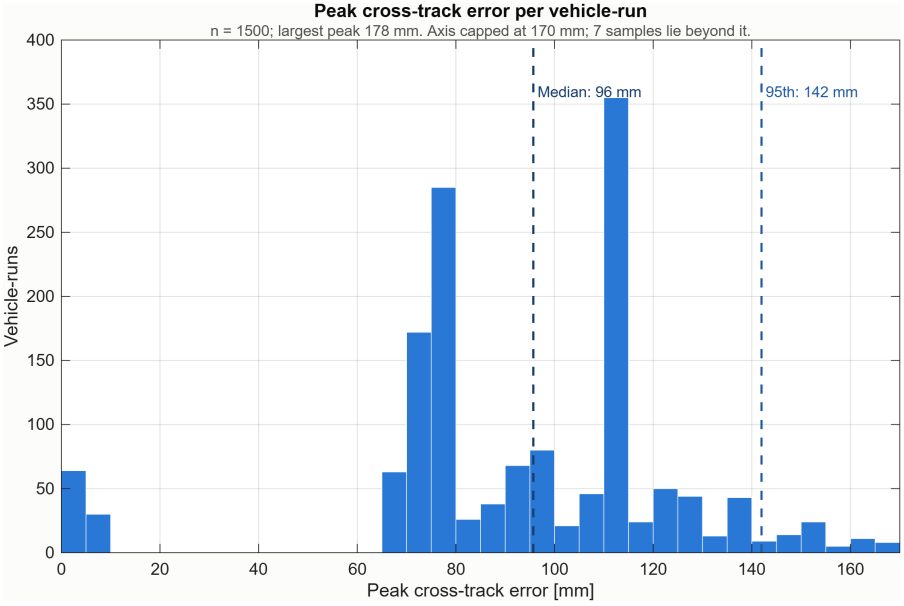


Figure 10.4.1: Peak cross-track error of each vehicle-run

10.5 Convergence

Figure 10.5.1 shows how the estimates settle as trials accumulate. The below-requirement rate settles after about 200 trials, finishing at 66.2 % with a 95 % interval of 61.9 to 70.2 %, and the 95th-percentile cross-track error holds at 142 mm over the last 100 vehicle-runs, so 500 trials are enough for both. The collision rate converges more slowly because collisions are rare: its interval of 0.4 to 2.3 % is wide relative to the 1 % criterion, and narrowing it to about ±0.5 % would take roughly 1500 trials.

10.6 Worst-case scenario

Figure 10.6.1 shows the run with the smallest separation, a crossing conflict in which vehicles 1 and 2 came within 118 mm. Vehicle 1 travels from $(2.0, 0)$ to $(0, 0.4)$ m along a path with six 90° turns that passes through the destination of vehicle 2, which drives straight from $(0.8, 1.6)$ to $(0.8, 0)$ m. Their separation falls

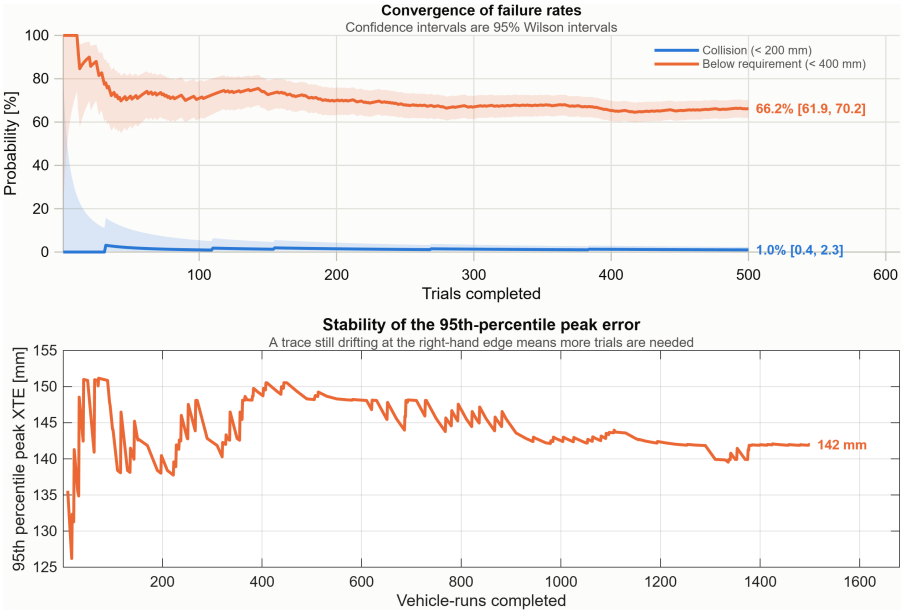


Figure 10.5.1: Convergence of the failure rates and of the 95th-percentile cross-track error

below 400 mm at 4.4 s, reaches its minimum at 6.7 s, as vehicle 2 arrives at its destination, and recovers at 8.6 s. The other pairs stay well clear, with minima of 783 and 1080 mm.

Two effects combine. Vehicle 1 cuts each corner, so it covers its path faster than planned and reaches vehicle 2's destination at about 6.7 s rather than the planned 8.0 s. Unlike the square test, where every turn goes the same way and the vehicle ends slightly late, each jog shortens the driven path, so this along-track error grows with the number of jogs. Meanwhile, vehicle 2 overshoots its destination by about 80 mm and vehicle 1 dips about 90 mm below the edge of the area, so both leave the movement area at the conflict point. Pure Pursuit corrects neither effect, because it controls only cross-track error. Penalising jogs in the planner, or controlling each vehicle's progress along its path, would address the cause.

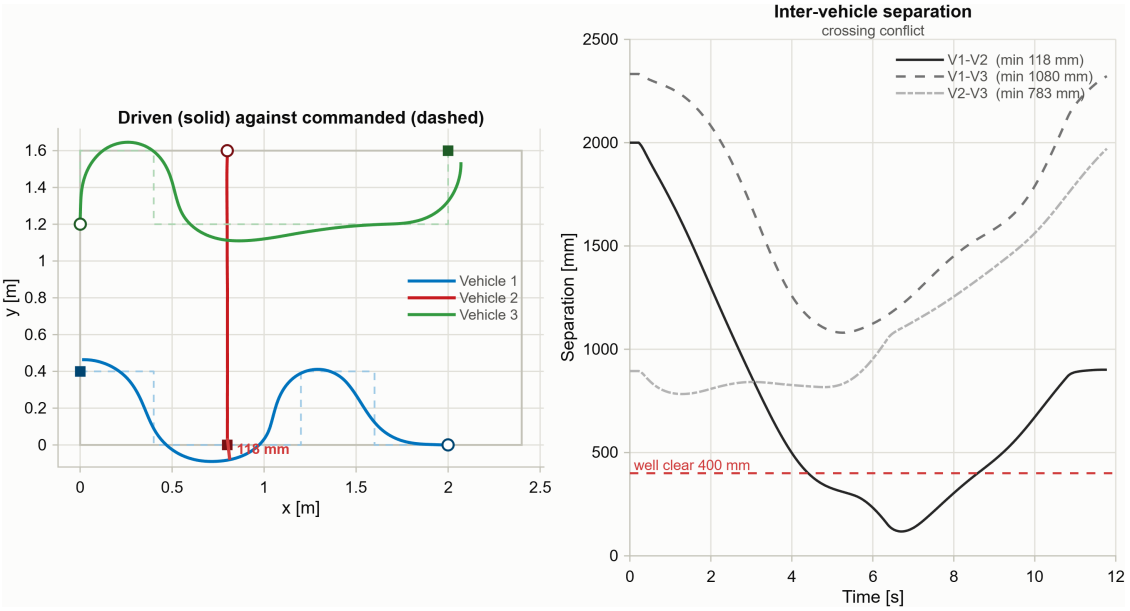


Figure 10.6.1: Worst-case run: driven and commanded paths (left) and inter-vehicle separation (right)

Appendix A
A* Search algorithm

A.1 Well-clear distance formula derivation

Over one timestep $[t_0, t_1]$ each vehicle travels in a straight line at constant speed. Writing $\tau = (t - t_0)/(t_1 - t_0) \in [0, 1]$ for normalised time, the two positions are linearly related in τ:

$$\mathbf{p}_a(\tau) = \mathbf{a}_0 + \tau(\mathbf{a}_1 - \mathbf{a}_0), \qquad \mathbf{p}_b(\tau) = \mathbf{b}_0 + \tau(\mathbf{b}_1 - \mathbf{b}_0). \tag{A.1.1}$$

Their difference is therefore also linear:

$$\mathbf{r}(\tau) = \mathbf{p}_a - \mathbf{p}_b = (\mathbf{a}_0 - \mathbf{b}_0) + \tau\big[(\mathbf{a}_1 - \mathbf{b}_1) - (\mathbf{a}_0 - \mathbf{b}_0)\big] = \mathbf{r}_0 + \tau\,\Delta\mathbf{r}, \tag{A.1.2}$$

with $\mathbf{r}_0 = \mathbf{a}_0 - \mathbf{b}_0$, $\mathbf{r}_1 = \mathbf{a}_1 - \mathbf{b}_1$ and $\Delta\mathbf{r} = \mathbf{r}_1 - \mathbf{r}_0$.

Minimising $\|\mathbf{r}\|$ is equivalent to minimising $D(\tau) = \|\mathbf{r}(\tau)\|^2$:

$$D(\tau) = \|\Delta\mathbf{r}\|^2\tau^2 + 2\,(\mathbf{r}_0 \cdot \Delta\mathbf{r})\,\tau + \|\mathbf{r}_0\|^2. \tag{A.1.3}$$

If $\Delta\mathbf{r} = \mathbf{0}$ the two velocities are equal, D is constant, and any τ serves; take $\tau^* = 0$. Otherwise $\|\Delta\mathbf{r}\|^2 > 0$, so D is a strictly convex quadratic with

$$D'(\tau) = 2\,(\mathbf{r}_0 \cdot \Delta\mathbf{r}) + 2\tau\,\|\Delta\mathbf{r}\|^2, \qquad D''(\tau) = 2\,\|\Delta\mathbf{r}\|^2 > 0, \tag{A.1.4}$$

and unconstrained minimiser

$$\hat{\tau} = -\frac{\mathbf{r}_0 \cdot \Delta\mathbf{r}}{\|\Delta\mathbf{r}\|^2}. \tag{A.1.5}$$

(Equivalently, $\hat{\tau}$ is the value for which $\mathbf{r}(\tau) \perp \Delta\mathbf{r}$, the foot of the perpendicular from the origin to the line.) Strict convexity makes D decreasing on $(-\infty, \hat{\tau}]$ and increasing on $[\hat{\tau}, \infty)$, so on the closed interval $[0, 1]$ the minimum lies at $\hat{\tau}$ if $\hat{\tau} \in [0, 1]$ and at the nearer endpoint otherwise. All cases are covered by

$$\tau^* = \mathrm{clamp}(\hat{\tau},\, 0,\, 1), \qquad d_{\min} = \big\|\mathbf{r}_0 + \tau^*\Delta\mathbf{r}\big\|, \tag{A.1.6}$$

and the pair is well clear over the step if and only if $d_{\min} \geq R_{ab}$. The clamp is the exact solution of the constrained problem, not a safeguard: convexity excludes any other minimiser on the interval.

Piecewise trajectories - When a one-step move is checked against an already-reserved multi-step trajectory, Equation A.1.1 holds only between that trajectory's waypoints. Partitioning the shared interval at those waypoints makes both vehicles linearly related on each piece, so Equation A.1.6 applies piece by piece; since the minimum over a finite union of intervals is the least of the minima over them, taking the least result is still exact.

Appendix B

Dynamic straight line model derivation

Taking moments about the vehicle centre C in the dynamic free-body diagram, with the two tractive forces acting at a moment arm $L = L_{track}/2$,

$$\sum T = J\alpha : \qquad J_{eff}\,\dot{\omega}_v = F_L L - F_R L - b_{eff}\,\omega_v \qquad \Longrightarrow \qquad J_{eff}\,\dot{\omega}_v + b_{eff}\,\omega_v = L\,(F_L - F_R). \tag{B.0.1}$$

Straight-line motion is the case $F_L = F_R$, for which the right-hand side vanishes and $\omega_v \to 0$: the yaw equation decouples and the vehicle reduces to a single translational degree of freedom. Resolving along x,

$$\sum F = ma : \qquad m_v\ddot{x} = 2F - b\dot{x} \qquad \Longrightarrow \qquad m_v\ddot{x} + b\dot{x} = 2F = 2\left(\frac{T}{r}\right), \tag{B.0.2}$$

each wheel converting motor torque T into tractive force T/r at the contact patch. The motor delivers $T = NKI$ through the gearbox, so

$$m_v\ddot{x} + b\dot{x} = 2\left(\frac{NK}{r}\right) I, \tag{B.0.3}$$

and with the armature inductance neglected the electrical loop gives $I = (E_{in} - KN\omega_m)/R_a$, whence

$$m_v\ddot{x} + b\dot{x} = 2\left(\frac{NK}{r}\right)\left(\frac{E_{in} - KN\omega_m}{R_a}\right) = \frac{2NK}{rR_a}\big(E_{in} - KN\omega_m\big). \tag{B.0.4}$$

Rolling without slip relates the translational and wheel states by $\dot{x} = r\omega$ and $\ddot{x} = r\dot{\omega}$, so

$$m_v r\dot{\omega} + br\omega = \frac{2NK}{rR_a}\big(E_{in} - KN\omega_m\big), \tag{B.0.5}$$

and dividing through by r,

$$m_v\dot{\omega} + b\omega = \frac{2NK}{r^2R_a}\big(E_{in} - KN\omega_m\big). \tag{B.0.6}$$

Taking the Laplace transform with zero initial conditions, and with $\omega_m = N\omega$ so that the back-EMF term becomes $KN\Omega$,

$$\mathcal{L}\{\ \}: \qquad m_v s\,\Omega + b\,\Omega = \frac{2NK}{r^2 R_a}\big(E - KN\Omega\big). \tag{B.0.7}$$

Expanding the right-hand side and gathering the Ω terms on the left,

$$m_v s\,\Omega + \left[\, b + \frac{2N^2K^2}{r^2 R_a} \,\right]\Omega = \frac{2NK}{r^2 R_a}\,E, \tag{B.0.8}$$

the back-EMF having contributed an additional damping term alongside b. Multiplying through by $r^2 R_a$ to clear the denominators,

$$m_v r^2 R_a\, s\,\Omega + \big[\, br^2 R_a + 2N^2K^2 \,\big]\,\Omega = 2NK\,E, \tag{B.0.9}$$

and solving for the ratio gives the plant transfer function from input voltage to wheel angular velocity,

$$G(s) = \frac{\Omega}{E} = \frac{2NK}{m_v r^2 R_a\, s + \big(br^2 R_a + 2N^2K^2\big)}. \tag{B.0.10}$$

This is first order, of the standard form $K_{dc}/(\tau s + 1)$ with

$$K_{dc} = \frac{2NK}{br^2 R_a + 2N^2K^2}, \qquad \tau = \frac{m_v r^2 R_a}{br^2 R_a + 2N^2K^2}, \tag{B.0.11}$$

so a step of amplitude E from rest produces $\omega(t) = K_{dc}E\,(1 - e^{-t/\tau})$. Both parameters are obtainable directly from a step test without evaluating N, K, R_a or b individually: the steady-state wheel speed gives $K_{dc} = \omega_{ss}/E$, and the 10%–90% rise time of a first-order response is $t_r = \tau \ln 9$, so $\tau = t_r/\ln 9$. Forward speed follows as $\dot{x} = r\omega$.

The vehicle measurements are a track width of $L_{track} = 136$ mm centre to centre, giving a moment arm $L = 68$ mm in Equation B.0.1, and a wheel diameter of 65 mm, giving $r = 32.5$ mm.

The derivation assumes rolling without slip, a rigid chassis, negligible armature inductance and purely viscous friction. The last is the most consequential: the drivetrain exhibits Coulomb and static friction, giving a dead band at low command that Equation B.0.10 cannot represent and which is characterised separately.

List of References

Alarifi, A., Al-Salman, A., Alsaleh, M., Alnafessah, A., Al-Hadhrami, S., Al-Ammar, M.A. and Al-Khalifa, H.S. (2016). Ultra wideband indoor positioning technologies: Analysis and recent advances. *Sensors*, vol. 16, no. 5, p. 707.

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Coulter, R.C. (1992). Implementation of the pure pursuit path tracking algorithm. Tech. Rep. CMU-RI-TR-92-01, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA.

Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271.

Duchoň, F., Babinec, A., Kajan, M., Beňo, P., Florek, M., Fico, T. and Jurišica, L. (2014). Path planning with modified a star algorithm for a mobile robot. *Procedia Engineering*, vol. 96, pp. 59–69.

Fox, D., Burgard, W. and Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33.

Franklin, G.F., Powell, J.D. and Emami-Naeini, A. (2019). *Feedback Control of Dynamic Systems*. 8th edn. Pearson.

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F.J. and Marín-Jiménez, M.J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292.

Ge, S.S. and Cui, Y.J. (2000). New potential functions for mobile robot path planning. *IEEE Transactions on Robotics and Automation*, vol. 16, no. 5, pp. 615–620.

Hafner, M.R., Cunningham, D., Caminiti, L. and Del Vecchio, D. (2013). Cooperative collision avoidance at intersections: Algorithms and experiments. *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1162–1175.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107.

Karaman, S. and Frazzoli, E. (2011). Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894.

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98.

Kluever, C.A. (2019). *Dynamic Systems: Modeling, Simulation, and Control*. 2nd edn. John Wiley & Sons.

Koenig, S. and Likhachev, M. (2002). D* lite. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 476–483.

LaValle, S.M. (1998). Rapidly-exploring random trees: A new tool for path planning. Tech. Rep. TR 98-11, Computer Science Department, Iowa State University.

Merriaux, P., Dupuis, Y., Boutteau, R., Vasseur, P. and Savatier, X. (2017). A study of vicon system positioning performance. *Sensors*, vol. 17, no. 7, p. 1591.

Mukhtar, A., Xia, L. and Tang, T.B. (2015). Vehicle detection techniques for collision avoidance systems: A review. *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 5, pp. 2318–2338.

Olson, E. (2011). Apriltag: A robust and flexible visual fiducial system. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3400–3407.

Patle, B.K., Ganesh Babu, L., Pandey, A., Parhi, D.R.K. and Jagadeesh, A. (2019). A review: On path planning strategies for navigation of mobile robot. *Defence Technology*, vol. 15, no. 4, pp. 582–606.

Romero-Ramirez, F.J., Muñoz-Salinas, R. and Medina-Carnicer, R. (2018). Speeded up detection of squared fiducial markers. *Image and Vision Computing*, vol. 76, pp. 38–47.

Silver, D. (2005). Cooperative pathfinding. In: *Proceedings of the First AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pp. 117–122.

Stentz, A. (1994). Optimal and efficient path planning for partially-known environments. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3310–3317.

Vahidi, A. and Eskandarian, A. (2003). Research advances in intelligent collision avoidance and adaptive cruise control. *IEEE Transactions on Intelligent Transportation Systems*, vol. 4, no. 3, pp. 143–153.

van den Berg, J., Guy, S.J., Lin, M. and Manocha, D. (2011). Reciprocal n-body collision avoidance. In: *Robotics Research: The 14th International Symposium ISRR*, vol. 70 of *Springer Tracts in Advanced Robotics*, pp. 3–19. Springer.

van den Berg, J., Lin, M. and Manocha, D. (2008). Reciprocal velocity obstacles for real-time multi-agent navigation. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1928–1935.

Wang, J. and Olson, E. (2016). Apriltag 2: Efficient and robust fiducial detection. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4193–4198.

Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334.